



IT DEPARTMENT

END OF STUDIES PROJECT REPORT

In partial fulfillment of the requirements for the Bachelor's degree in:

Ingénierie Des Systèmes Informatiques

Systèmes Embarqués et Internet Des Objets

Booking Simulation

Prepared by: **Lourimi Ayoub**

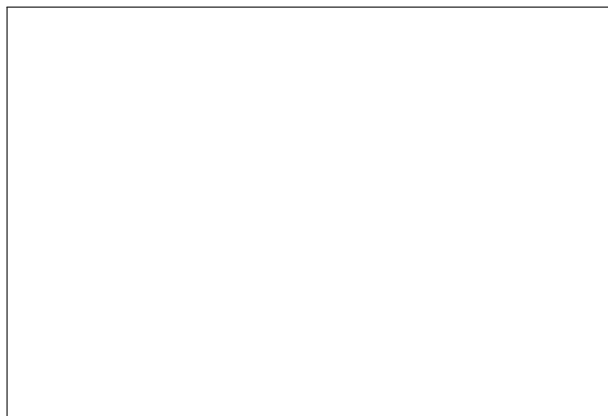
Supervised by:

Academic supervisor : Mr. Ali Lourimi, ISSATSo

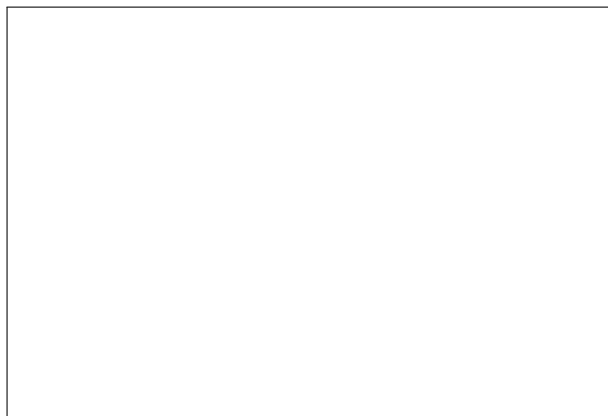
Professional supervisor : Mr. Haroun Bouriga, Smarteo



SIGNATURES OF SUPERVISORS



Mr. Haroun bouriga



Mr. Ali lourimi

I dedicate this work

First and foremost, to God for the strength He gave me, the faith He strengthened within me, and His constant presence by my side, not only during the completion of this thesis but throughout every stage that led me to this moment.

To my dear parents for the trust they have always placed in me, the unwavering support they provided, and the valuable guidance they offered. They have taught me values of uniqueness, determination, and ambition. I am deeply grateful and proud to have them as my parents.

To my dear brother and sister for being constant pillars of support, always providing me with motivation and strength in times of uncertainty and challenges. My trust in them is total and unshakable.

To my precious friends for their constant presence, sincere support, and encouragement during key moments of this journey. Their friendship has been a true source of strength and motivation.

I sincerely thank all the teaching staff of the **Higher Institute of Applied Sciences and Technology of Sousse** as well as the professional contributors for their guidance and the richness of their teaching.

To my academic supervisor, Mr. Ali LOURIMI, whose expertise, wisdom, and valuable advice greatly enriched my research, and whose exceptional mentorship was a precious support throughout my internship.

To my professional supervisor, Mr. Haroun BOURIGA, I would like to express my deepest gratitude for his valuable advice and support throughout my final year internship. His trust in my abilities and constant encouragement were essential to my professional development.

Sincerely,
Lourimi Ayoub

Contents

List of Acronyms	7
General Introduction	8
1 General Project Presentation	9
1.1 Introduction	10
1.2 Company Presentation	10
1.2.1 Company Activities	10
1.3 Problem Statement	11
1.4 Project Presentation	11
1.4.1 System Architecture	12
1.4.2 Project Objectives	12
1.4.3 Main Features	12
1.4.4 Technologies Used	13
1.5 State of the Art	13
1.5.1 Frontend Technologies	13
1.5.2 Backend Technologies	13
1.5.3 Geospatial Routing and Simulation	13
1.5.4 Database	13
1.5.5 Containerization	14
1.6 Development Process	14
1.6.1 Justification of the Methodological Choice	14
1.6.2 Comparison with Other Methodologies	14
1.6.3 V-Model Overview	14
1.6.4 Development Phases	14
1.6.5 Validation Phases	15
1.7 Conclusion	15
2 Project Specification	16
2.1 Introduction	17
2.2 Project Vision	17
2.2.1 Identification of Functional Requirements	17
2.2.2 Identification of Non-Functional Requirements	18
2.3 Specification of Requirements	19
2.3.1 Identification of Actors	19
2.3.2 Use Case Diagram	19
2.3.3 Description of Use Cases	20
2.4 Detailed Requirements Specification	21
2.4.1 Textual Description of Major Use Cases	21
2.4.2 System Sequence Diagrams of Major Use Cases	26

3	Design	32
3.1	Introduction	33
3.2	Sequence Diagrams	33
3.2.1	Authentication Sequence Diagram	33
3.2.2	Agent Authentication Sequence Diagram	35
3.2.3	Passenger Ride Request and Dispatch Sequence Diagram	37
3.2.4	Driver Accept Ride Sequence Diagram	39
3.2.5	Real-Time Driver Tracking Sequence Diagram	41
3.2.6	Admin/Agent Manage Drivers Sequence Diagram	42
3.3	Conclusion	44
4	Implementation	45
4.1	Introduction	46
4.2	Working Environment	46
4.2.1	Hardware Environment	46
4.2.2	Software Environment	46
4.2.2.1	Design Tools	46
4.2.2.2	Development Tools	47
4.2.2.3	Version Control and Collaboration Tools	47
4.2.2.4	Testing Tools	47
4.2.2.5	Report Writing Tools	47
4.2.2.6	Programming Languages and Frameworks	48
4.2.2.7	APIs and Libraries Used	49
4.2.2.8	Database	50
4.3	System Architecture	50
4.3.1	Physical Architecture	50
4.3.2	Logical Architecture	51
4.4	Test Cases (Experimental Results)	52
4.4.1	Introduction	52
4.4.2	Passenger Interface	52
4.4.3	Driver Interface	53
4.4.4	Ride Request Notification Interface	55
4.4.5	Admin / Agent Dashboard	55
4.4.6	Analytics Dashboard Interface	57
4.4.7	Real-Time Tracking Interface	58
4.5	Conclusion	58
	General Conclusion	59

List of Tables

1.1	Comparison between Classical, Agile, and V-Model Approaches	14
2.1	Actors and their functionalities in the Taxi Booking System	21
2.2	Textual description of the use case "Authentication"	22
2.3	Textual description of the use case "Request a Ride"	22
2.4	Textual description of the use case "Respond to Ride Request"	23
2.5	Textual description of the use case "Confirm Booking"	23
2.6	Textual description of the use case "Track Driver"	24
2.7	Textual description of the use case "Cancel Booking"	25
2.8	Textual description of the use case "End Ride"	25
2.9	Textual description of the use case "Manage Drivers"	26
3.1	Description of the Sequence Diagram "Authentication"	35
3.2	Description of the Sequence Diagram "Agent Authentication"	36
3.3	Description of the Sequence Diagram "Passenger Ride Request and Dispatch" . .	39
3.4	Description of the Sequence Diagram "Driver Accept Ride"	40
3.5	Description of the Sequence Diagram "Real-Time Driver Tracking"	42
3.6	Description of the Sequence Diagram "Manage Drivers"	43
4.1	Description of the hardware environment	46
4.2	Design Tools	46
4.3	Development Tools	47
4.4	Version Control and Collaboration Tools	47
4.5	Testing Tools	47
4.6	Report Writing Tools	48
4.7	Programming Languages and Frameworks	48
4.8	APIs and Libraries Used	49
4.9	Database	50

List of Figures

1.1	Smarteo	10
1.2	V-Model	15
2.1	Use Case Diagram of the Taxi Booking System	20
2.2	Use-case diagram: Authentication	21
2.3	Use-case diagram: Request a Ride	22
2.4	Use-case diagram: Respond to Ride Request	23
2.5	Use-case diagram: Confirm Booking	23
2.6	Use-case diagram: Track Driver	24
2.7	Use-case diagram: Cancel Booking	25
2.8	Use-case diagram: End Ride	25
2.9	Use-case diagram: Manage Drivers	26
2.10	System Sequence Diagram: Request Ride	27
2.11	System Sequence Diagram: Respond to Ride Request	28
2.12	System Sequence Diagram: Confirm Booking	28
2.13	System Sequence Diagram: Track Driver	29
2.14	System Sequence Diagram: Cancel Booking	29
2.15	System Sequence Diagram: End Ride	30
2.16	System Sequence Diagram: Manage Drivers	31
3.1	Sequence Diagram: Authentication	34
3.2	Sequence Diagram: Agent Authentication	36
3.3	Sequence Diagram: Passenger Ride Request and Dispatch	38
3.4	Sequence Diagram: Driver Accept Ride	40
3.5	Sequence Diagram: Real-Time Driver Tracking	41
3.6	Sequence Diagram: Manage Drivers	43
4.1	Layered (n-tier) architecture of the system	51
4.2	Logical architecture of the system	51
4.3	Passenger Home Interface	53
4.4	Driver Dashboard Interface	54
4.5	Driver Ride Request Notification	55
4.6	Driver Management Interface	56
4.7	Analytics Dashboard Interface	57
4.8	Real-Time Driver Tracking Interface	58

List of Acronyms

API Application Programming Interface

UI User Interface

UX User Experience

DB Database

NoSQL Not Only SQL

JWT JSON Web Token

REST Representational State Transfer

OSRM Open Source Routing Machine

GPS Global Positioning System

SDK Software Development Kit

CRUD Create, Read, Update, Delete

Socket.IO Real-time communication library based on WebSockets

PFE Projet de Fin d'Études

GENERAL INTRODUCTION

In a context where digital transformation is deeply reshaping transportation and mobility systems, intelligent mobility solutions have become increasingly essential. Real-time taxi booking applications significantly improve urban transport efficiency by optimizing the connection between passengers and drivers through smart and responsive systems.

However, the implementation of such systems introduces several technical challenges, including real-time geolocation management, coordination between distributed actors, and event synchronization in a highly dynamic environment. These constraints make such systems both complex to design and difficult to fully understand, especially in an educational context.

In this framework, this final year project (PFE) is carried out in collaboration with Smarteo. The main objective is to design and develop a real-time taxi booking simulation platform capable of reproducing realistic urban mobility scenarios while providing a clear visualization of the system's internal behavior.

This platform goes beyond a simple booking application. It simulates the complete lifecycle of a ride, from the passenger request to trip completion, including driver assignment, real-time communication, and live tracking on an interactive map. It also integrates advanced simulation mechanisms to test different dispatching and routing scenarios.

As part of this final year project, conducted at the Higher Institute of Applied Science and Technology of Sousse (ISSATSo) for the academic year 2025/2026, we developed a solution based on modern web and mobile technologies, ensuring a scalable and real-time architecture.

The purpose of this report is to present in a structured manner the different phases of design and implementation of the project. It covers the general context, problem statement, system requirements, technological choices, and achieved results.

The report is organized as follows:

- **Chapter 1:** General presentation of the project, the hosting company Smarteo, and the context and problem definition of the taxi simulation system.
- **Chapter 2:** Specification of functional and non-functional requirements of the system.
- **Chapter 3:** System design study, including the system architecture and UML modeling of the proposed solution.
- **Chapter 4:** Implementation of the application and presentation of the developed interfaces and main functionalities.

The report concludes with a general conclusion and future improvements and possible enhancements of the project.

General Project Presentation

Table of Contents

1.1	Introduction	10
1.2	Company Presentation	10
1.2.1	Company Activities	10
1.3	Problem Statement	11
1.4	Project Presentation	11
1.4.1	System Architecture	12
1.4.2	Project Objectives	12
1.4.3	Main Features	12
1.4.4	Technologies Used	13
1.5	State of the Art	13
1.5.1	Frontend Technologies	13
1.5.2	Backend Technologies	13
1.5.3	Geospatial Routing and Simulation	13
1.5.4	Database	13
1.5.5	Containerization	14
1.6	Development Process	14
1.6.1	Justification of the Methodological Choice	14
1.6.2	Comparison with Other Methodologies	14
1.6.3	V-Model Overview	14
1.6.4	Development Phases	14
1.6.5	Validation Phases	15
1.7	Conclusion	15

1.1 Introduction

This chapter aims to present the general context of the project. It introduces the end-of-studies project (PFE) environment by describing the company Smarteo, its services, and the environment in which the project was carried out. An overview of the existing solution is provided in order to identify its limitations and better understand the needs addressed by the application. Finally, the development methodology adopted for the project is presented.

1.2 Company Presentation

The project was developed in the context of Smarteo, a Tunisian company founded in 2019 and specialized in information systems and electronics.

Smarteo focuses on delivering integrated digital solutions combining both software and hardware components to support digital transformation.



Figure 1.1: Smarteo

1.2.1 Company Activities

Smarteo places customer satisfaction at the core of its priorities and considers it a key commitment. The company ensures high-quality services and continuous support throughout project development.

Its main services include:

- **Design and Study:** requirements analysis, solution design, and project planning.
- **Development:** implementation of hardware and software solutions.
- **Deployment:** integration, testing, and operational monitoring.

1.3 Problem Statement

With the rise of digital technologies and intelligent transport systems, real-time taxi booking applications play a major role in urban mobility. These systems allow passengers to quickly connect with drivers while optimizing routes, waiting time, and resource usage.

However, such systems are technically complex. A real-time taxi booking system relies on several critical components such as:

- real-time geolocation management,
- instant communication between actors,
- dynamic decision-making for taxi assignment,
- booking state management (waiting, accepted, in progress, completed).

Existing solutions are often complex and difficult to understand, especially for educational purposes. It becomes challenging for students or beginners to understand:

- taxi assignment logic,
- real-time communication flow,
- frontend-backend synchronization.

Moreover, most tools do not provide practical simulation scenarios such as:

- taxi rejection,
- multi-taxi search before acceptance,
- booking cancellation,
- full route tracking on an interactive map.

Therefore, there is a need for a clear and interactive system to simulate and visualize the booking process.

Thus, this project aims to design and develop a web application capable of realistically simulating a real-time taxi booking system in an interactive and educational manner.

1.4 Project Presentation

The project consists of designing and developing a real-time taxi booking simulation system that connects multiple clients and services through a centralized backend architecture.

The system supports both real users and simulation clients to reproduce realistic transportation scenarios.

1.4.1 System Architecture

The architecture follows a multi-client real-time model composed of:

- **Backend Server:** Node.js with Express and Socket.IO for core business logic
- **Passenger Mobile App:** React Native application for ride requests and tracking
- **Driver Mobile App:** React Native application for receiving and managing ride assignments
- **Web Simulation Client:** React (Vite) application for testing and simulation scenarios
- **Shared Layer:** Common Socket.IO utilities and communication standards

The system also integrates external services:

- OSRM for route computation and travel time estimation
- Optional AI-based dispatch optimization using external intelligent services

1.4.2 Project Objectives

The main objectives of this project are:

- Simulate a complete real-time taxi booking lifecycle
- Enable communication between passengers and drivers using WebSockets
- Provide accurate route computation using map services
- Support multiple simulation scenarios for testing and evaluation
- Offer a visual and interactive educational tool

1.4.3 Main Features

The system includes the following functionalities:

- Real-time ride request and dispatch
- Driver acceptance and rejection mechanism
- Live tracking of vehicles on map interfaces
- Booking lifecycle management (waiting, accepted, in progress, completed, cancelled)
- Scenario-based simulation and stress testing
- Administrative management of drivers and system state

1.4.4 Technologies Used

The project is built using modern web and mobile technologies:

- **Backend:** Node.js, Express, Socket.IO, MongoDB, JWT authentication
- **Mobile:** React Native (Expo), TypeScript, Mapbox
- **Web Client:** React, Vite, Leaflet, PrimeReact, Tailwind CSS
- **Routing:** OSRM API for real-world path computation
- **AI Component:** Optional dispatch optimization using external AI services

1.5 State of the Art

Modern taxi booking systems depend on advanced technologies to ensure real-time communication, efficient data management, and a smooth user experience.

1.5.1 Frontend Technologies

React.js was chosen for the frontend due to:

- Component reusability,
- Efficient UI management,
- Virtual DOM performance optimization.

Leaflet is used for interactive map management.

1.5.2 Backend Technologies

The backend is built using Node.js, Express, and Socket.IO to ensure real-time communication:

- Booking request handling,
- Real-time driver notifications,
- Vehicle tracking updates,
- Event management.

1.5.3 Geospatial Routing and Simulation

The Open Source Routing Machine (OSRM) is integrated into the system in order to compute realistic road-based routes, distances, and estimated travel times. This integration enhances the accuracy of the simulation and improves the realism of driver navigation and dispatch scenarios.

1.5.4 Database

MongoDB is used as a NoSQL database due to its flexibility, scalability, and compatibility with Node.js.

1.5.5 Containerization

Docker is used to containerize services, ensuring portability, isolation, and simplified deployment.

1.6 Development Process

A structured methodology is essential to ensure system quality and validation. The project follows the V-Model, a sequential development approach where each development phase corresponds to a testing phase.

1.6.1 Justification of the Methodological Choice

The V-Model was selected due to its structured nature, ensuring a clear correspondence between development and testing phases. This makes it particularly suitable for academic projects requiring strong documentation, traceability, and validation.

1.6.2 Comparison with Other Methodologies

The table below presents a comparison between the main software development methodologies.

TABLE 1.1 – Comparison between Classical, Agile, and V-Model Approaches

Criteria	Classical Model	Agile (Scrum)	V-Model
Process Type	Linear and sequential	Iterative and incremental	Sequential with validation mapping
Flexibility	Low	High	Medium
Documentation	Very high	Moderate	High
Testing Phase	At the end	Continuous	Paired with each phase
Suitability	Stable requirements	Changing requirements	Well-defined academic projects
Project Control	Strict	Flexible	Structured and controlled

The V-Model provides an appropriate balance between structure, documentation, and verification, making it suitable for this project.

1.6.3 V-Model Overview

The V-Model follows a top-down development approach and a bottom-up testing approach.

1.6.4 Development Phases

- Requirements analysis
- System design
- Module design
- Implementation

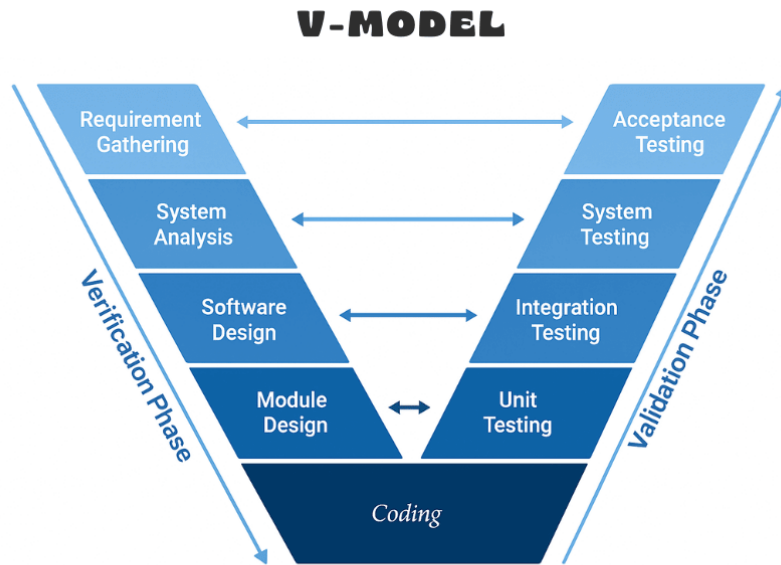


Figure 1.2: V-Model

1.6.5 Validation Phases

- Unit testing
- Integration testing
- System testing
- Acceptance testing

1.7 Conclusion

This chapter presented the general context of the project. We introduced the company, analyzed the problem, and defined the objectives of the system.

We also studied existing solutions and technologies used in modern taxi systems. Finally, we presented the V-Model methodology adopted for the development process, ensuring a structured and reliable system design.

Project Specification

Table of Contents

2.1	Introduction	17
2.2	Project Vision	17
2.2.1	Identification of Functional Requirements	17
2.2.2	Identification of Non-Functional Requirements	18
2.3	Specification of Requirements	19
2.3.1	Identification of Actors	19
2.3.2	Use Case Diagram	19
2.3.3	Description of Use Cases	20
2.4	Detailed Requirements Specification	21
2.4.1	Textual Description of Major Use Cases	21
2.4.2	System Sequence Diagrams of Major Use Cases	26

2.1 Introduction

This chapter defines the specification of the proposed system. It presents the project vision and details both functional and non-functional requirements.

These specifications describe what the system is expected to achieve, as well as the constraints under which it must operate. This ensures a clear understanding of the system before moving to the design phase.

2.2 Project Vision

The main vision of this project is to design and implement a real-time taxi booking simulation platform capable of reproducing realistic ride-hailing scenarios.

The system enables interaction between multiple actors (passengers, drivers, and administrators) through a real-time architecture based on event-driven communication. It focuses on simulating the complete lifecycle of a ride, from request creation to trip completion, while ensuring transparency, traceability, and real-time synchronization across all components.

2.2.1 Identification of Functional Requirements

The proposed system provides a set of functional requirements that define the main services offered to the users. These functionalities are designed to simulate a realistic taxi booking environment.

User Authentication and Role Management

The system requires all users (passengers, drivers, and administrators) to authenticate before accessing their respective interfaces. This ensures secure access and allows the system to differentiate functionalities based on user roles.

Each role has specific permissions:

- ✓ Passengers can request rides and track trips.
- ✓ Drivers can receive and manage ride assignments.
- ✓ Administrators can monitor and manage the system state.

Ride Request Management

Passengers are able to create ride requests in real time by selecting a destination on the map. Once a request is submitted, it is transmitted instantly to the backend system, which processes it and initiates the driver assignment process.

This feature simulates a real-world ride-hailing behavior where requests are handled dynamically based on driver availability and proximity.

Driver Assignment and Ride Handling

The system is responsible for assigning available drivers to incoming ride requests. Drivers receive notifications in real time and can either accept or reject a ride.

Once a ride is accepted, the system updates the booking state and begins tracking the trip until completion.

Real-Time Tracking and Communication

The platform provides real-time tracking of drivers on an interactive map. Passengers can observe the movement of the assigned vehicle and follow the progression of the trip.

This functionality is supported by WebSocket communication, ensuring continuous synchronization between backend and clients.

Booking Lifecycle Management

The system manages the complete lifecycle of a booking, which includes the following states:

- ✓ Waiting (request sent)
- ✓ Accepted (driver assigned)
- ✓ In progress (trip started)
- ✓ Completed (trip finished)
- ✓ Cancelled (request rejected or aborted)

Each state transition is handled in real time to ensure consistency across all connected clients.

Administrative and Simulation Features

Administrators have access to a control interface that allows monitoring drivers, managing system states, and executing simulation scenarios.

The system also supports testing scenarios such as driver rejection, multiple search attempts, and controlled simulation modes for educational purposes.

2.2.2 Identification of Non-Functional Requirements

In addition to functional requirements, the system must satisfy several non-functional requirements that ensure quality, reliability, and usability.

Performance

The system must guarantee low latency communication between clients and server. Real-time updates such as ride requests and driver movements must be processed instantly to ensure a smooth user experience, even under multiple concurrent users.

Scalability

The architecture is designed to support multiple simultaneous users, including passengers and drivers, without degradation in performance. This ensures that the system can evolve from a simulation environment to a larger-scale deployment if needed.

Security

Security is a critical aspect of the system. Authentication is handled using secure JWT-based mechanisms, ensuring that only authorized users can access system functionalities. Passwords and sensitive data are protected using encryption techniques.

Usability

The system provides intuitive and user-friendly interfaces for all actors. The goal is to ensure that users can easily interact with the platform without requiring technical knowledge, making it suitable for educational and demonstration purposes.

Reliability and Availability

The system must remain stable and available during continuous simulation sessions. It should be able to handle unexpected disconnections or errors without affecting the overall system behavior.

Maintainability and Extensibility

The architecture follows a modular design, allowing easy maintenance and future extension. New features such as advanced dispatching algorithms or additional user roles can be integrated without major modifications to the system.

2.3 Specification of Requirements

This section presents the actors of the system and the corresponding use case diagram. It describes how different users interact with the taxi booking simulation platform.

2.3.1 Identification of Actors

The system is composed of four main actors, each having specific responsibilities and access rights:

- **Passenger:** Represents the end-user who requests a ride, tracks the driver, and manages reservations.
- **Driver:** Represents the taxi driver responsible for receiving ride requests, updating location, and managing trip execution.
- **Agent:** Represents a system operator responsible for managing drivers, monitoring system activity, and accessing analytics.
- **Admin:** Represents the highest-level user who manages agents and system permissions. The Admin inherits all Agent functionalities.

2.3.2 Use Case Diagram

The following use case diagram illustrates the interactions between the system actors and the different functionalities offered by the taxi booking platform.



Figure 2.1: Use Case Diagram of the Taxi Booking System

2.3.3 Description of Use Cases

The following table summarizes the main roles of the system and their associated functionalities.

Role	Functions
Passenger	A passenger is an end user who requests transportation services through the platform. The passenger can search for a destination, request a ride, join a queue, confirm or cancel a reservation, and track the assigned driver in real time.
Driver	A driver is responsible for receiving and managing ride requests. The driver can receive reservation offers, respond by accepting or declining a ride, update real-time location, pick up passengers, and complete trips.
Agent	An agent is a system operator responsible for monitoring and managing the fleet. The agent can manage drivers, verify accounts, block or unblock drivers, assign taxis, and view system analytics.
Administrator	The administrator is the highest-level user responsible for managing the platform. The administrator manages agents, assigns permissions, and oversees system-wide operations.

TABLE 2.1 – Actors and their functionalities in the Taxi Booking System

2.4 Detailed Requirements Specification

2.4.1 Textual Description of Major Use Cases

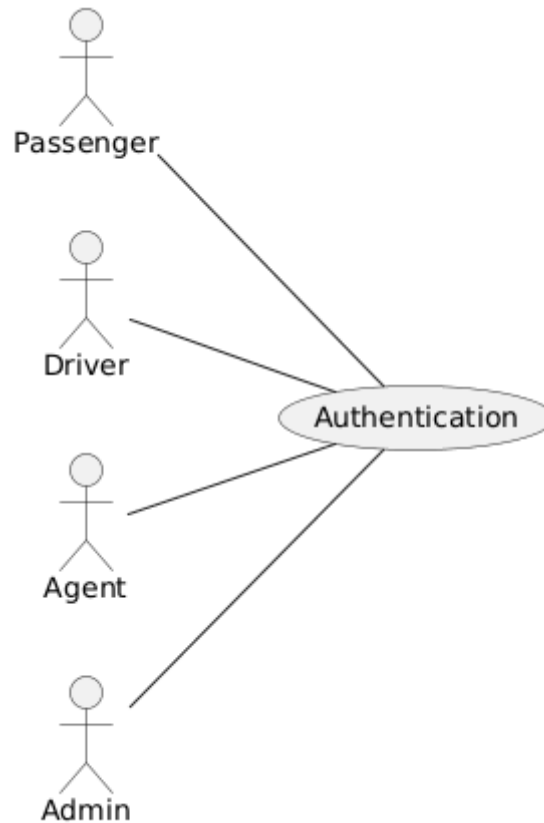


Figure 2.2: Use-case diagram: Authentication

TABLE 2.2 – Textual description of the use case "Authentication"

Use Case	Authentication
Actor	Passenger, Driver, Admin, Agent
Preconditions	The user must have a valid account on the platform.
Main Scenario	1. The user accesses the login page. 2. The system displays the authentication form. 3. The user enters email and password. 4. The user submits the login request. 5. The system verifies credentials. 6. The system authenticates the user. 7. The system redirects the user according to their role.
Alternative Scenario	1. Incorrect credentials are provided. 2. The system displays an authentication error message. 3. The user is prompted to try again.
Postconditions	• The user is successfully authenticated. • A secure session is created.

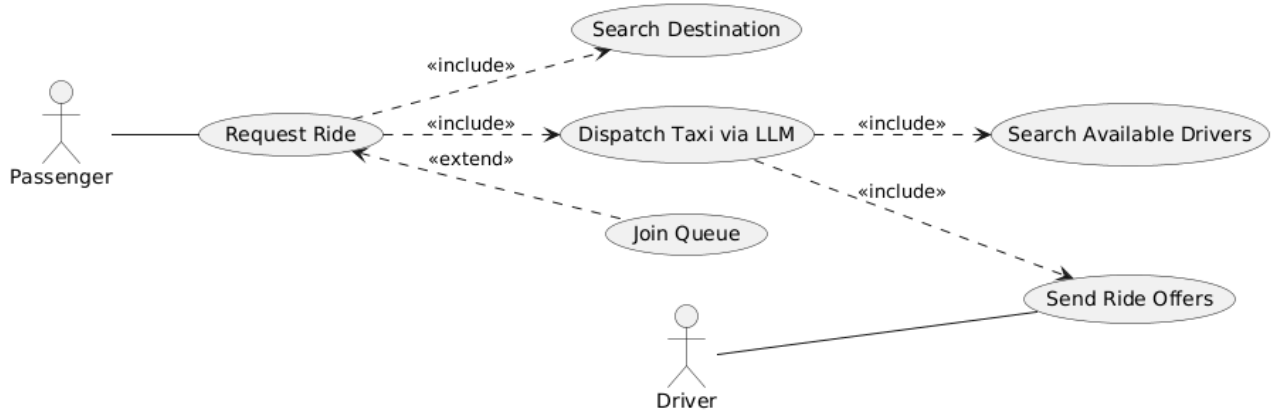


Figure 2.3: Use-case diagram: Request a Ride

TABLE 2.3 – Textual description of the use case "Request a Ride"

Use Case	Request a Ride
Actor	Passenger
Preconditions	The passenger is authenticated and connected to the system.
Main Scenario	1. The passenger selects a destination on the map. 2. The passenger submits a ride request. 3. The system processes the request. 4. The system searches for available drivers. 5. Ride offers are sent to drivers.

Use Case	Request a Ride
Alternative Scenario	1. No driver available → the passenger is placed in a waiting queue. 2. Invalid coordinates → an error message is displayed.
Postconditions	A booking is created with status <i>waiting</i> .

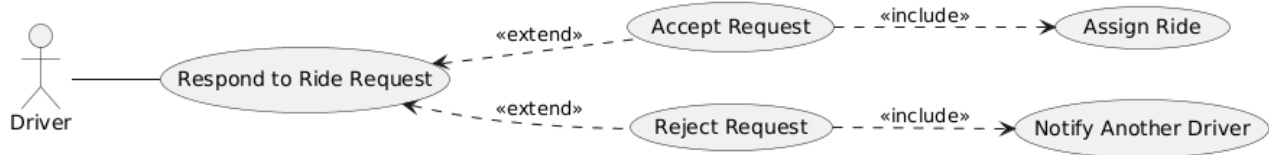


Figure 2.4: Use-case diagram: Respond to Ride Request

TABLE 2.4 – Textual description of the use case "Respond to Ride Request"

Use Case	Respond to Ride Request
Actor	Driver
Preconditions	The driver is authenticated and available.
Main Scenario	1. The driver receives a ride offer. 2. The driver reviews ride details. 3. The driver accepts the request. 4. The system assigns the ride.
Alternative Scenario	1. Driver rejects the request → another driver is notified. 2. No driver accepts → the request is cancelled.
Postconditions	The booking status becomes <i>accepted</i> .

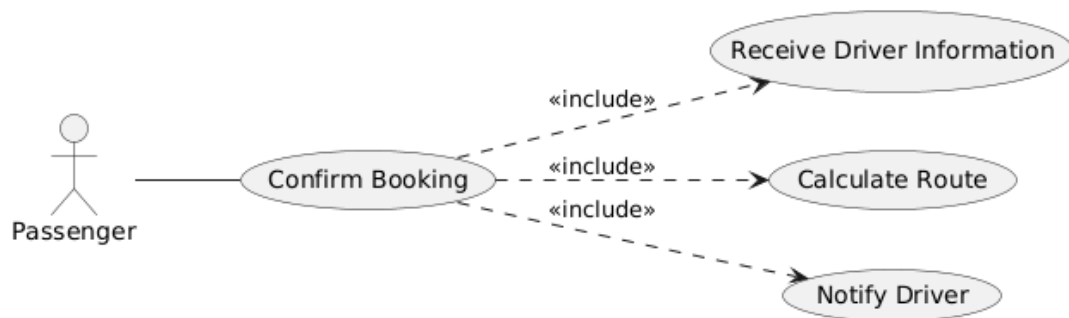


Figure 2.5: Use-case diagram: Confirm Booking

TABLE 2.5 – Textual description of the use case "Confirm Booking"

Use Case	Confirm Booking
Actor	Passenger

Use Case	Confirm Booking
Preconditions	A driver has accepted the ride request.
Main Scenario	1. The passenger receives driver information. 2. The passenger confirms the booking. 3. The system calculates the route. 4. The driver is notified.
Alternative Scenario	1. No confirmation → booking may expire.
Postconditions	The booking status becomes <i>arriving</i> .

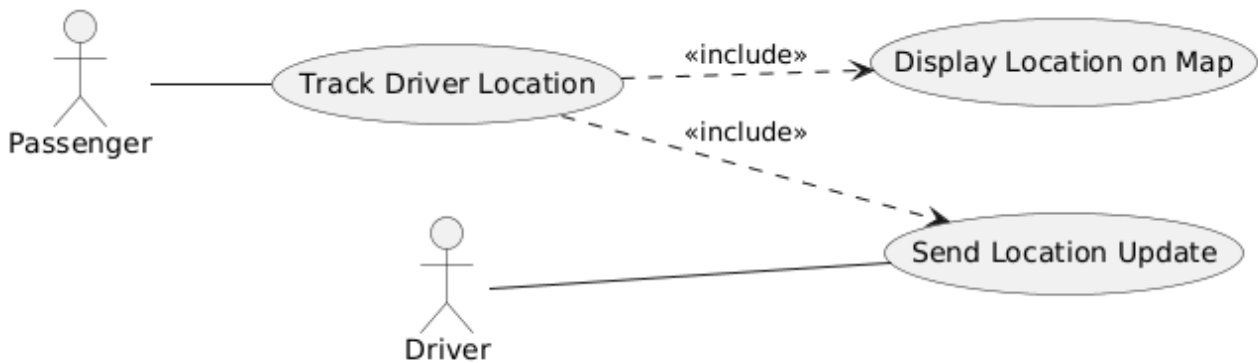


Figure 2.6: Use-case diagram: Track Driver

TABLE 2.6 – Textual description of the use case "Track Driver"

Use Case	Track Driver
Actor	Passenger
Preconditions	A booking is active.
Main Scenario	1. The system receives driver location updates. 2. The position is displayed on the map. 3. The passenger tracks the ride in real time.
Alternative Scenario	1. Connection loss → updates are interrupted.
Postconditions	Real-time tracking is enabled.

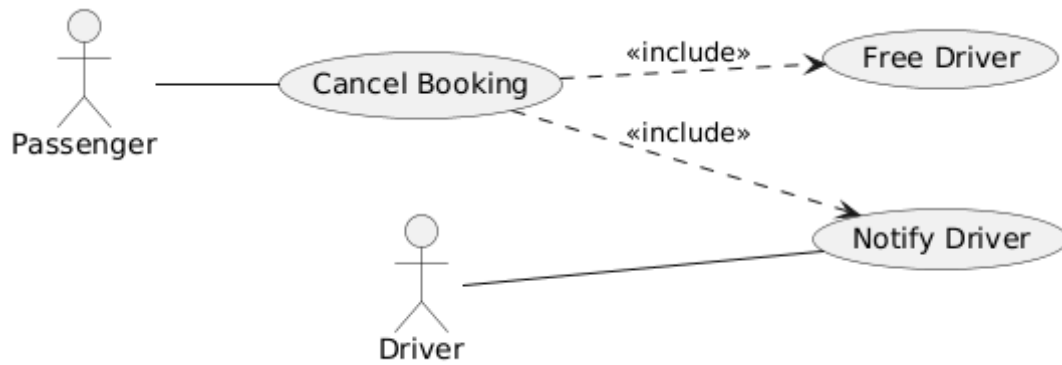


Figure 2.7: Use-case diagram: Cancel Booking

TABLE 2.7 – Textual description of the use case "Cancel Booking"

Use Case	Cancel Booking
Actor	Passenger
Preconditions	A booking exists.
Main Scenario	1. The passenger clicks "Cancel". 2. The system cancels the booking. 3. The driver is notified. 4. The system frees the driver.
Alternative Scenario	1. Trip already completed → cancellation is rejected.
Postconditions	The booking is <i>cancelled</i> .

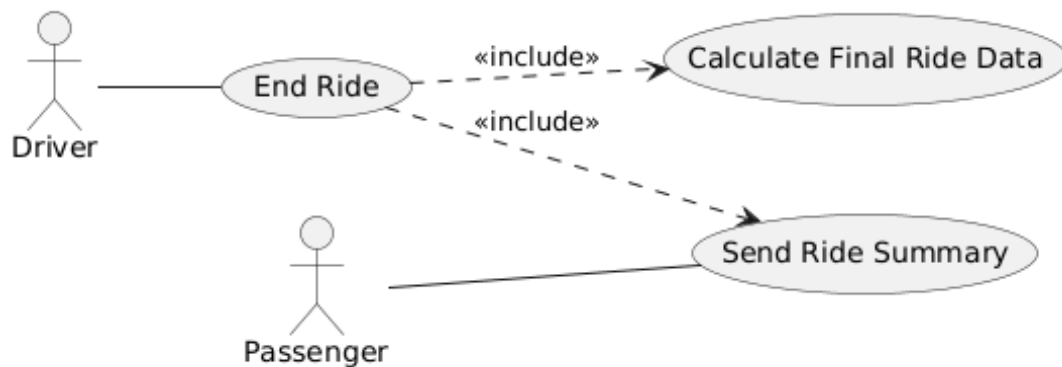


Figure 2.8: Use-case diagram: End Ride

TABLE 2.8 – Textual description of the use case "End Ride"

Use Case	End Ride
Actor	Driver
Preconditions	The ride is in progress.

Main Scenario	1. The driver reaches the destination. 2. The driver ends the ride. 3. The system calculates final ride data. 4. The passenger receives a summary.
Alternative Scenario	1. System error → fallback to available data.
Postconditions	The booking is completed and stored.

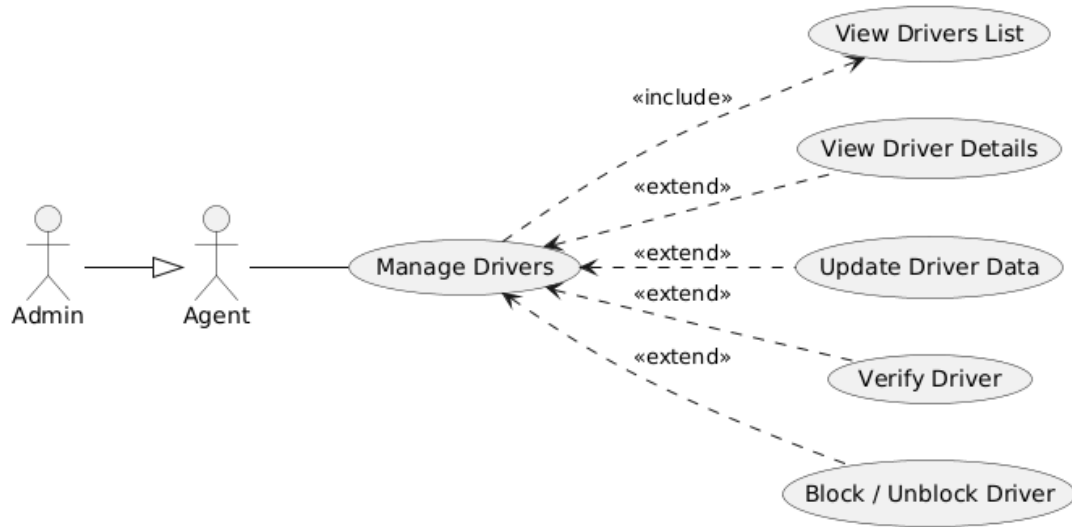


Figure 2.9: Use-case diagram: Manage Drivers

TABLE 2.9 – Textual description of the use case "Manage Drivers"

Use Case	Manage Drivers
Actor	Admin
Preconditions	The administrator is authenticated and has required permissions.
Main Scenario	1. The admin accesses the driver management panel. 2. The system displays the list of drivers. 3. The admin selects an action (view, update, verify, or block). 4. The system applies the requested action. 5. The system updates driver data.
Alternative Scenario	1. Invalid data → error message displayed. 2. System failure → action is not executed.
Postconditions	Driver information is updated and reflected in the system.

2.4.2 System Sequence Diagrams of Major Use Cases

This section presents the System Sequence Diagrams (SSD) of the most important use cases of the system. These diagrams illustrate the interactions between external actors (Passenger,

Driver, and Admin) and the system considered as a black box, without detailing its internal implementation.

SSD 1: Request Ride

This diagram illustrates the interaction between the passenger and the system when requesting a ride.

- The passenger sends a ride request with pickup and destination coordinates.
- The system validates the request.
- The system searches for available drivers.
- The system sends ride offers to nearby drivers.

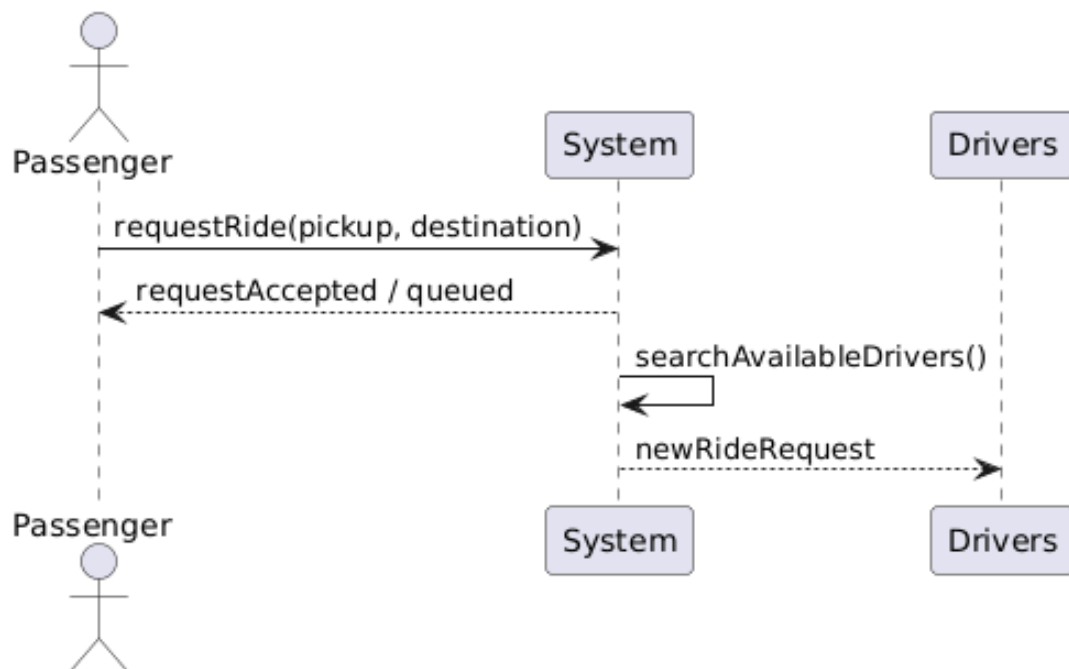


Figure 2.10: System Sequence Diagram: Request Ride

SSD 2: Respond to Ride Request

This diagram describes how a driver responds to a ride request.

- The system sends a ride request notification to the driver.
- The driver accepts or rejects the request.
- The system updates the booking status accordingly.

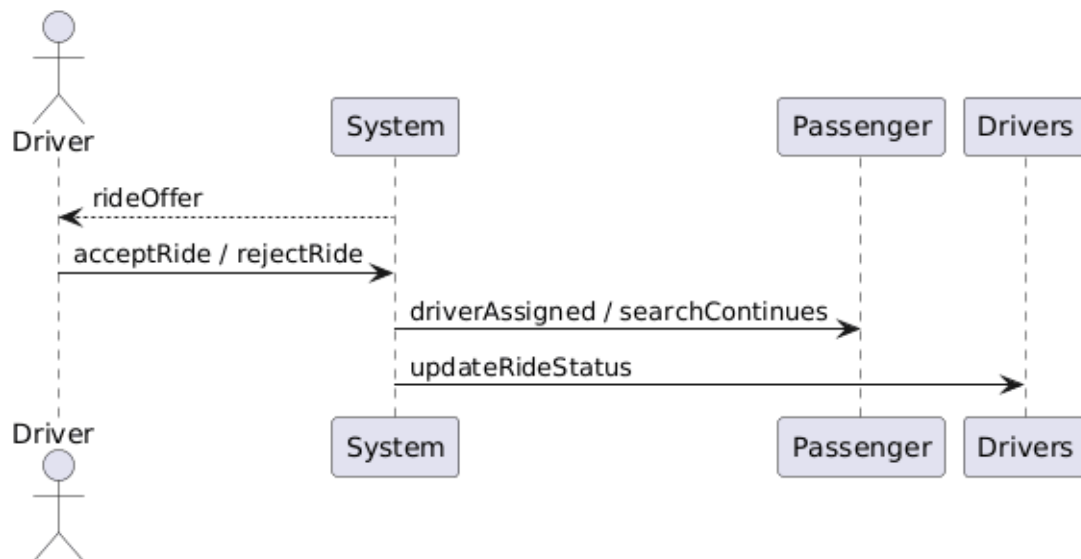


Figure 2.11: System Sequence Diagram: Respond to Ride Request

SSD 3: Confirm Booking

This diagram shows the booking confirmation process by the passenger.

- The passenger receives driver information.
- The passenger confirms the booking.
- The system calculates the route.
- The system notifies the driver.

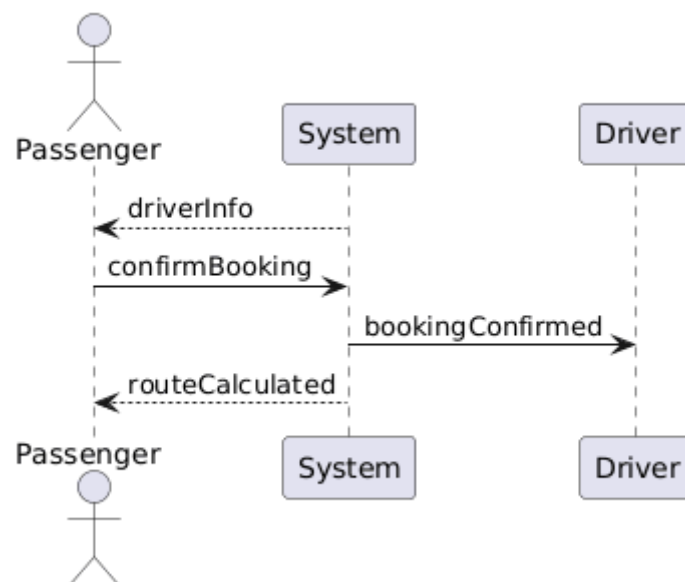


Figure 2.12: System Sequence Diagram: Confirm Booking

SSD 4: Track Driver

This diagram represents the real-time tracking of the driver.

- The driver sends periodic location updates.
- The system forwards updates to the passenger.
- The passenger visualizes the driver movement in real time.

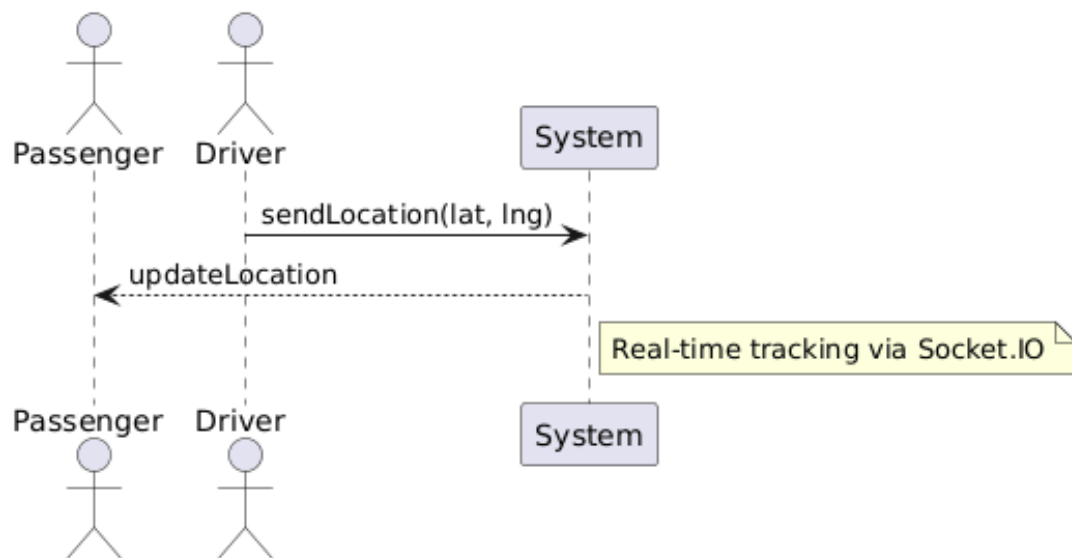


Figure 2.13: System Sequence Diagram: Track Driver

SSD 5: Cancel Booking

This diagram illustrates how a passenger cancels a booking.

- The passenger requests cancellation.
- The system cancels the booking.
- The system notifies the driver.
- The driver is released from the assignment.

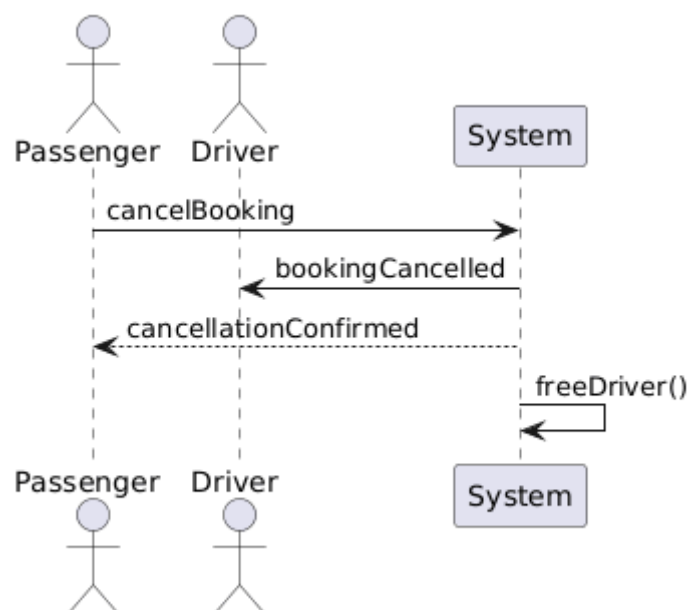


Figure 2.14: System Sequence Diagram: Cancel Booking

SSD 6: End Ride

This diagram describes the ride completion process.

- The driver marks the ride as completed.
- The system calculates final ride data.
- The system stores the trip information.
- The passenger receives a trip summary.

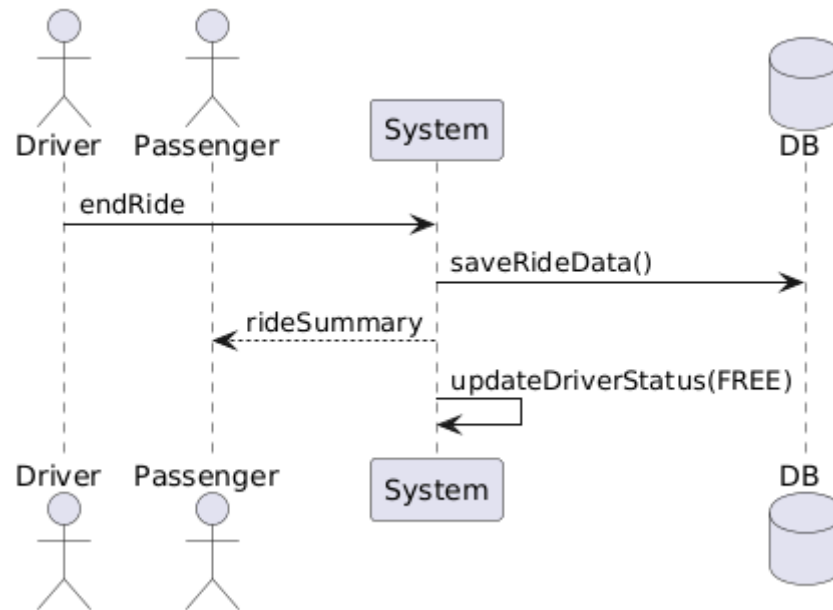


Figure 2.15: System Sequence Diagram: End Ride

SSD 7: Manage Drivers

This diagram shows how the administrator manages drivers.

- The administrator requests the list of drivers.
- The system displays driver information.
- The administrator performs management actions (verify, update, block).
- The system updates driver data accordingly.

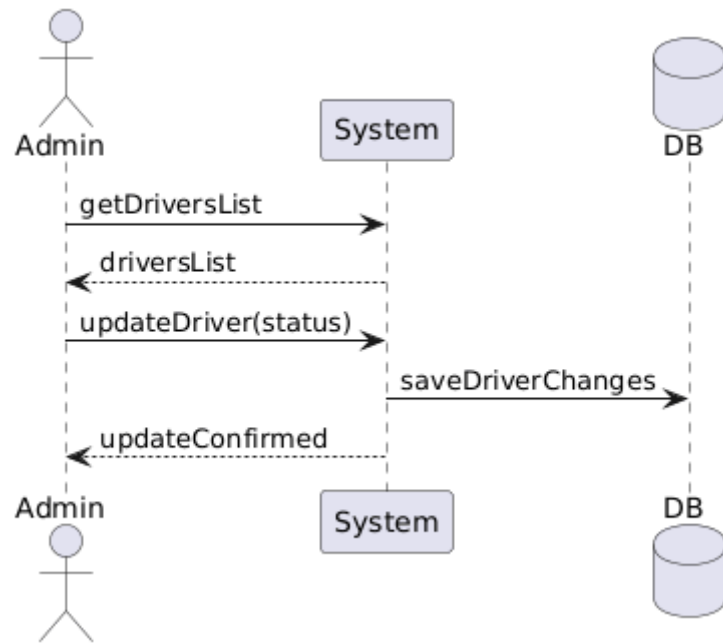


Figure 2.16: System Sequence Diagram: Manage Drivers

Conclusion

In this chapter, we defined the specification of the proposed system, including functional and non-functional requirements. We also identified the main actors and their interactions.

These elements provide a clear understanding of the system and serve as a basis for the next chapter, which focuses on system design.

Table of Contents

3.1	Introduction	33
3.2	Sequence Diagrams	33
3.2.1	Authentication Sequence Diagram	33
3.2.2	Agent Authentication Sequence Diagram	35
3.2.3	Passenger Ride Request and Dispatch Sequence Diagram	37
3.2.4	Driver Accept Ride Sequence Diagram	39
3.2.5	Real-Time Driver Tracking Sequence Diagram	41
3.2.6	Admin/Agent Manage Drivers Sequence Diagram	42
3.3	Conclusion	44

3.1 Introduction

This chapter presents the design phase of the proposed system, which represents a fundamental step in the software development lifecycle. First, we introduce the overall architecture of the application. Then, we describe the internal behavior of the system through a set of sequence diagrams and a class diagram.

3.2 Sequence Diagrams

Sequence diagrams are part of the Unified Modeling Language (UML). They are used to represent the chronological order of interactions between different system components during the execution of a specific scenario. These diagrams illustrate the exchanged messages between objects and provide a detailed understanding of the system behavior.

3.2.1 Authentication Sequence Diagram

The figure below presents the "Authentication" sequence diagram:

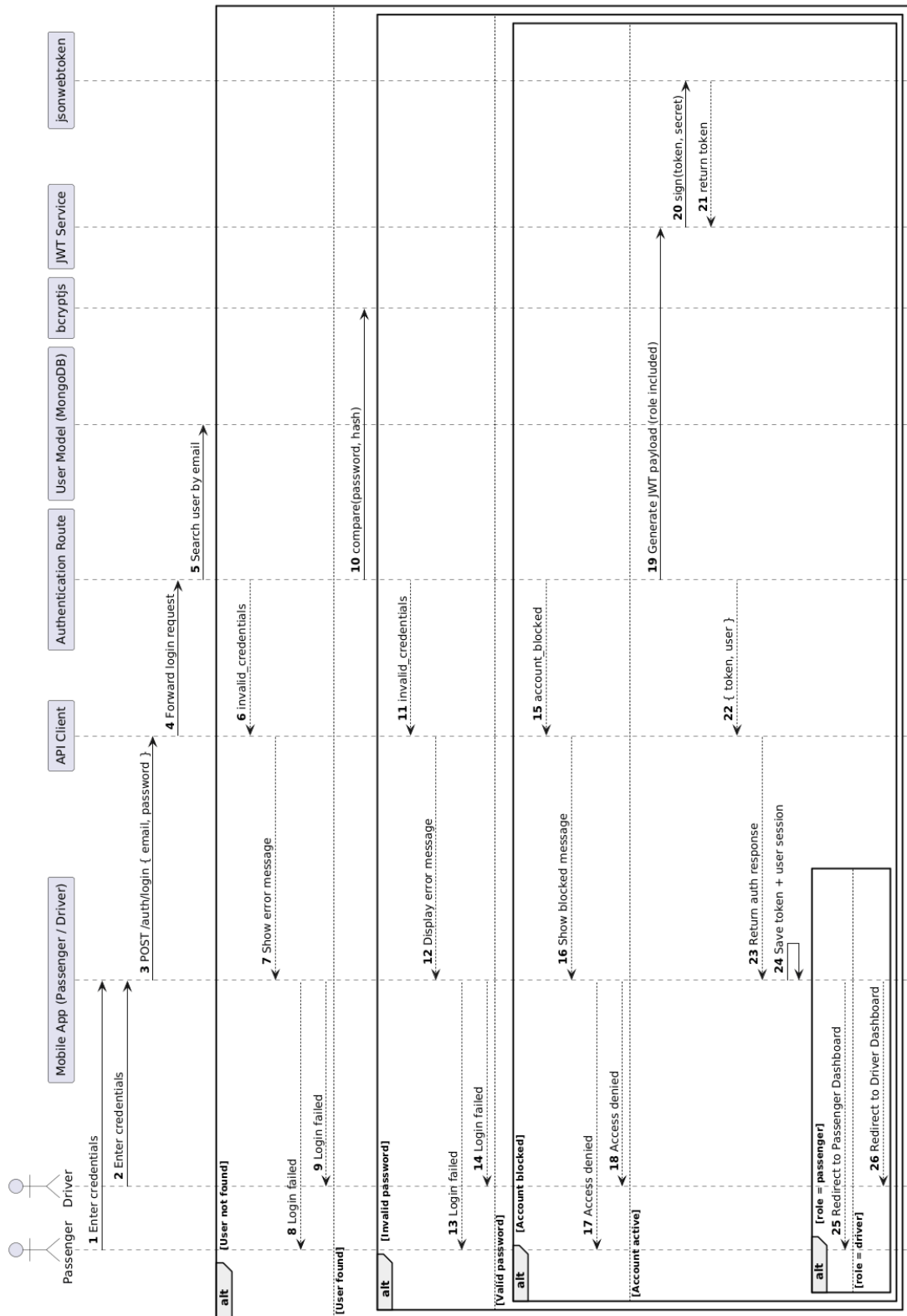


Figure 3.1: Sequence Diagram: Authentication

Description of the "Authentication" sequence diagram:

The table below describes the authentication process that passengers and drivers must follow before accessing system services:

TABLE 3.1 – Description of the Sequence Diagram "Authentication"

Title	Authentication
Summary	Passengers and drivers must authenticate themselves before accessing the application. The system verifies user credentials, validates account status, and generates a secure JWT session token according to the user role.
Actors	• Passenger • Driver
Preconditions	The login interface is displayed and the user already owns a valid account in the system.
Main Scenario	1. The passenger or driver enters email and password. 2. The mobile application sends a login request to the API client. 3. The authentication route receives and forwards the request. 4. The system searches for the user account in the database. 5. The system verifies the provided password using bcrypt. 6. The system checks whether the account is active or blocked. 7. The JWT service generates an authentication token containing the user role. 8. The token and user session information are returned to the mobile application. 9. The mobile application stores the session information locally. 10. The user is redirected to the corresponding dashboard according to the role.
Postconditions	• The user is successfully authenticated. • A secure JWT session is established. • The passenger or driver gains access to the application dashboard.

3.2.2 Agent Authentication Sequence Diagram

The figure below presents the "Agent Authentication" sequence diagram:

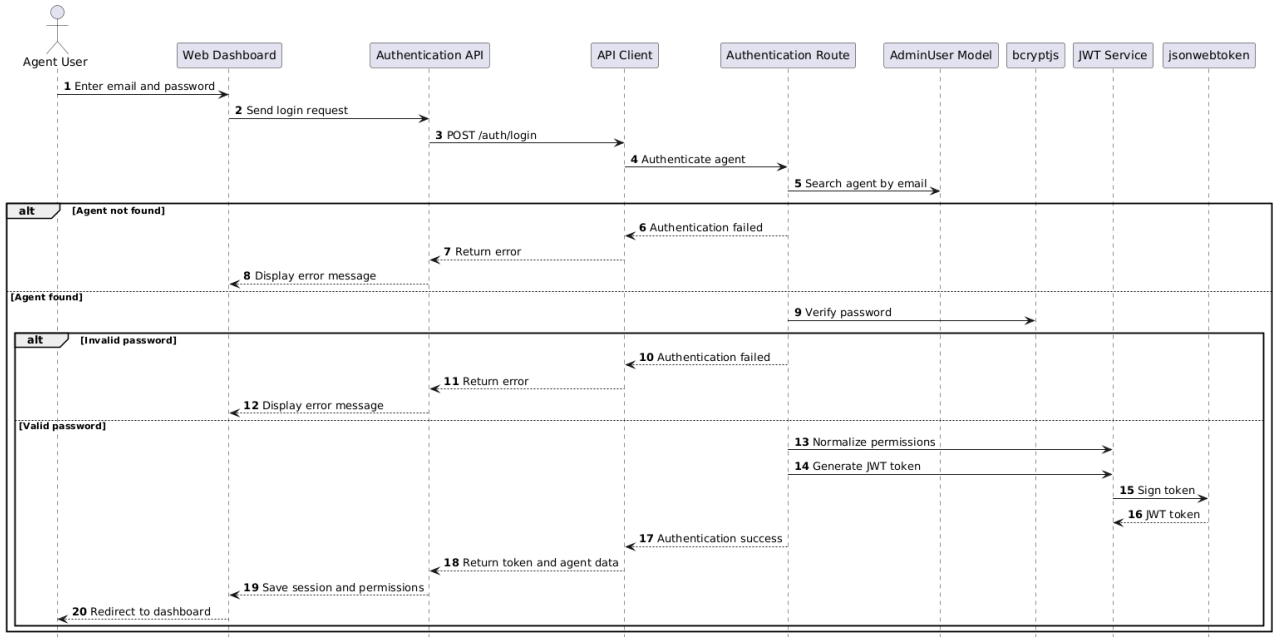


Figure 3.2: Sequence Diagram: Agent Authentication

Description of the "Agent Authentication" sequence diagram:

The table below describes the authentication process followed by the agent before accessing the administrative dashboard. The system verifies credentials, checks permissions, and generates a secure session token.

TABLE 3.2 – Description of the Sequence Diagram "Agent Authentication"

Title	Agent Authentication
Summary	Agents must authenticate themselves before accessing the administrative dashboard. The system validates credentials, verifies permissions, and generates a secure JWT session token for authorized access.
Actor	Agent
Preconditions	The login interface is displayed and the agent account already exists in the system.
Main Scenario	1. The agent enters email and password. 2. The web dashboard sends an authentication request to the API. 3. The authentication route searches for the agent account in the database. 4. The system verifies the password using bcrypt. 5. The system normalizes and validates the agent permissions. 6. The JWT service generates a secure authentication token. 7. The signed token and agent information are returned to the dashboard. 8. The dashboard stores the session and permissions locally. 9. The agent is redirected to the administration dashboard.

Title	Agent Authentication
Postconditions	• The agent is successfully authenticated. • A secure JWT session is established. • The agent gains access to the dashboard based on assigned permissions.

3.2.3 Passenger Ride Request and Dispatch Sequence Diagram

The figure below presents the "Passenger Ride Request and Dispatch" sequence diagram:

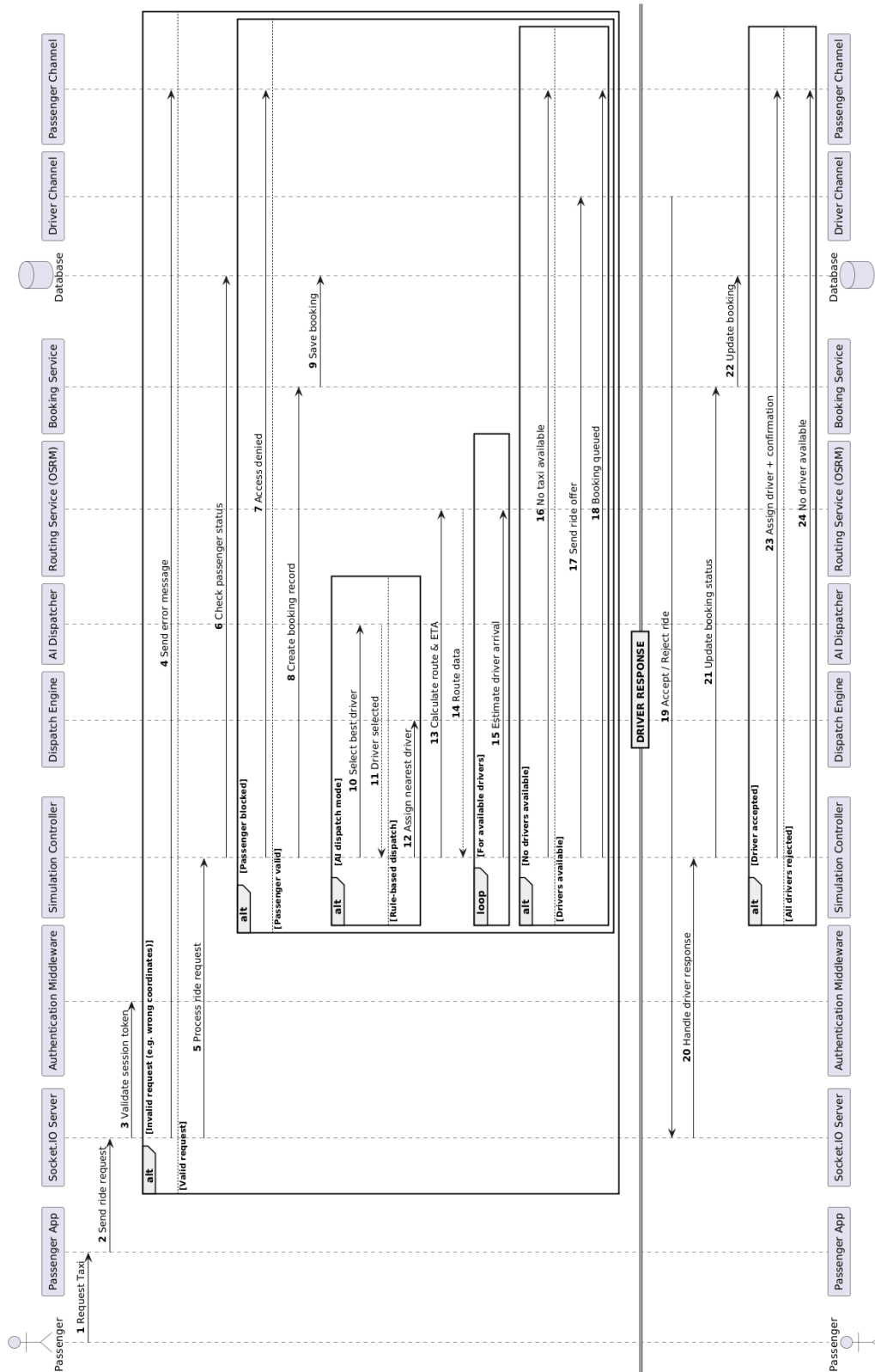


Figure 3.3: Sequence Diagram: Passenger Ride Request and Dispatch

Description of the "Passenger Ride Request and Dispatch" sequence diagram:

The table below describes the complete ride request workflow from the passenger side. The system processes the request in real time, assigns a driver using AI or rule-based dispatching, and manages booking updates through socket communication.

TABLE 3.3 – Description of the Sequence Diagram "Passenger Ride Request and Dispatch"

Title	Passenger Ride Request and Dispatch
Summary	This process describes how a passenger requests a ride, how the system validates and processes the request, and how a driver is assigned using either AI-based dispatching or a rule-based nearest driver strategy.
Actor	Passenger
Preconditions	The passenger is authenticated and connected to the system via the mobile application.
Main Scenario	1. The passenger requests a taxi through the mobile application. 2. The request is sent to the Socket.IO server. 3. The system validates the session and request data. 4. The simulation controller processes the ride request. 5. The system checks passenger status in the database. 6. A booking record is created and stored. 7. The system selects a driver using AI or rule-based dispatching. 8. The routing service calculates route and ETA. 9. Available drivers are notified of the ride request. 10. The passenger is informed that the request is being processed. 11. The system waits for driver response.
Alternative Scenario	1. No drivers available → the passenger is notified that no taxi is available. 2. Invalid request → the system returns an error message. 3. Passenger blocked → access is denied.
Postconditions	A booking is either assigned to a driver or marked as failed due to unavailability.

3.2.4 Driver Accept Ride Sequence Diagram

The figure below presents the "Driver Accept Ride" sequence diagram:

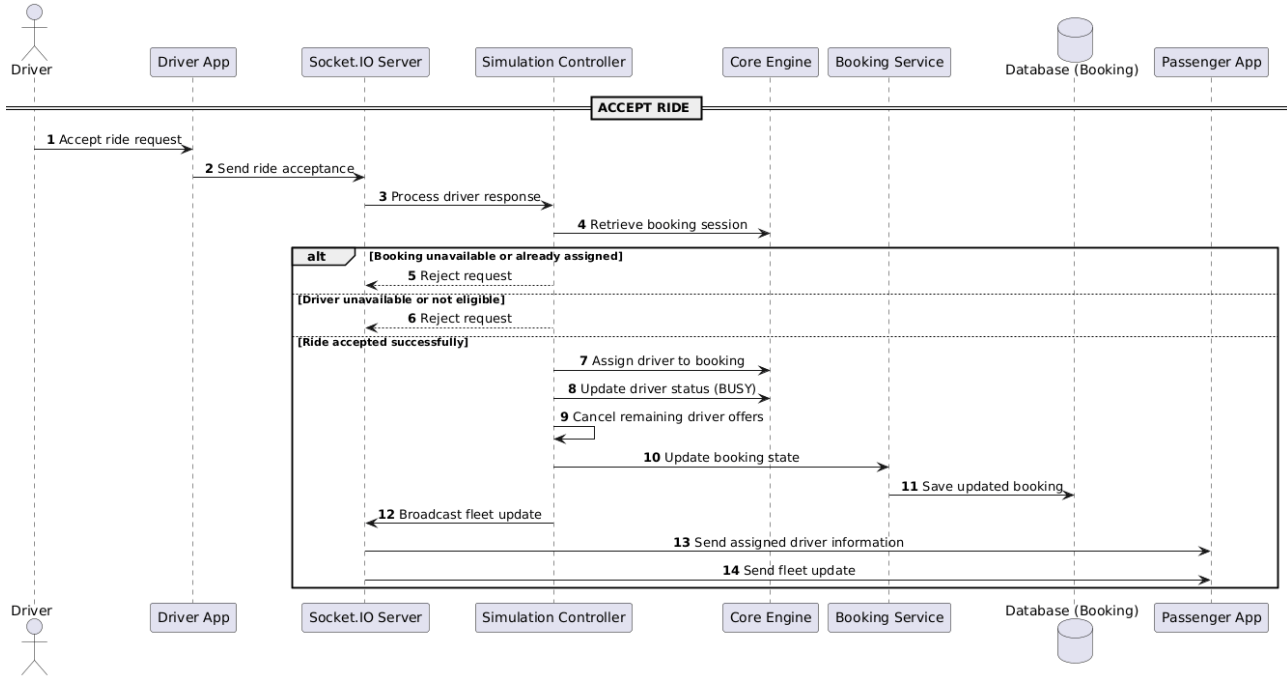


Figure 3.4: Sequence Diagram: Driver Accept Ride

Description of the "Driver Accept Ride" sequence diagram:

The table below describes the process followed when a driver accepts a ride request. The system validates the driver response in real time, assigns the booking if possible, updates the driver status, and notifies the passenger through Socket.IO communication.

TABLE 3.4 – Description of the Sequence Diagram "Driver Accept Ride"

Title	Driver Accept Ride
Summary	This sequence describes how a driver accepts a ride request. The system validates availability and eligibility, assigns the driver to the booking, updates the booking state, and notifies the passenger in real time.
Actor	Driver
Preconditions	A ride request has been sent and the driver is available to respond.
Main Scenario	1. The driver accepts the ride request from the mobile application. 2. The acceptance is sent to the Socket.IO server. 3. The simulation controller processes the driver response. 4. The system retrieves the booking session from the core engine. 5. The system validates booking availability. 6. The driver is assigned to the booking. 7. The driver status is updated to BUSY. 8. Remaining ride offers are canceled for other drivers. 9. The booking state is updated in the database. 10. The system broadcasts updates to connected clients. 11. The passenger receives driver assignment information.

Title	Driver Accept Ride
Alternative Scenario	1. Booking already assigned → request is rejected. 2. Driver not eligible → request is rejected. 3. Booking unavailable → request is rejected.
Postconditions	The booking is assigned to a driver and the system state is updated in real time.

3.2.5 Real-Time Driver Tracking Sequence Diagram

The figure below presents the "Real-Time Driver Tracking" sequence diagram:

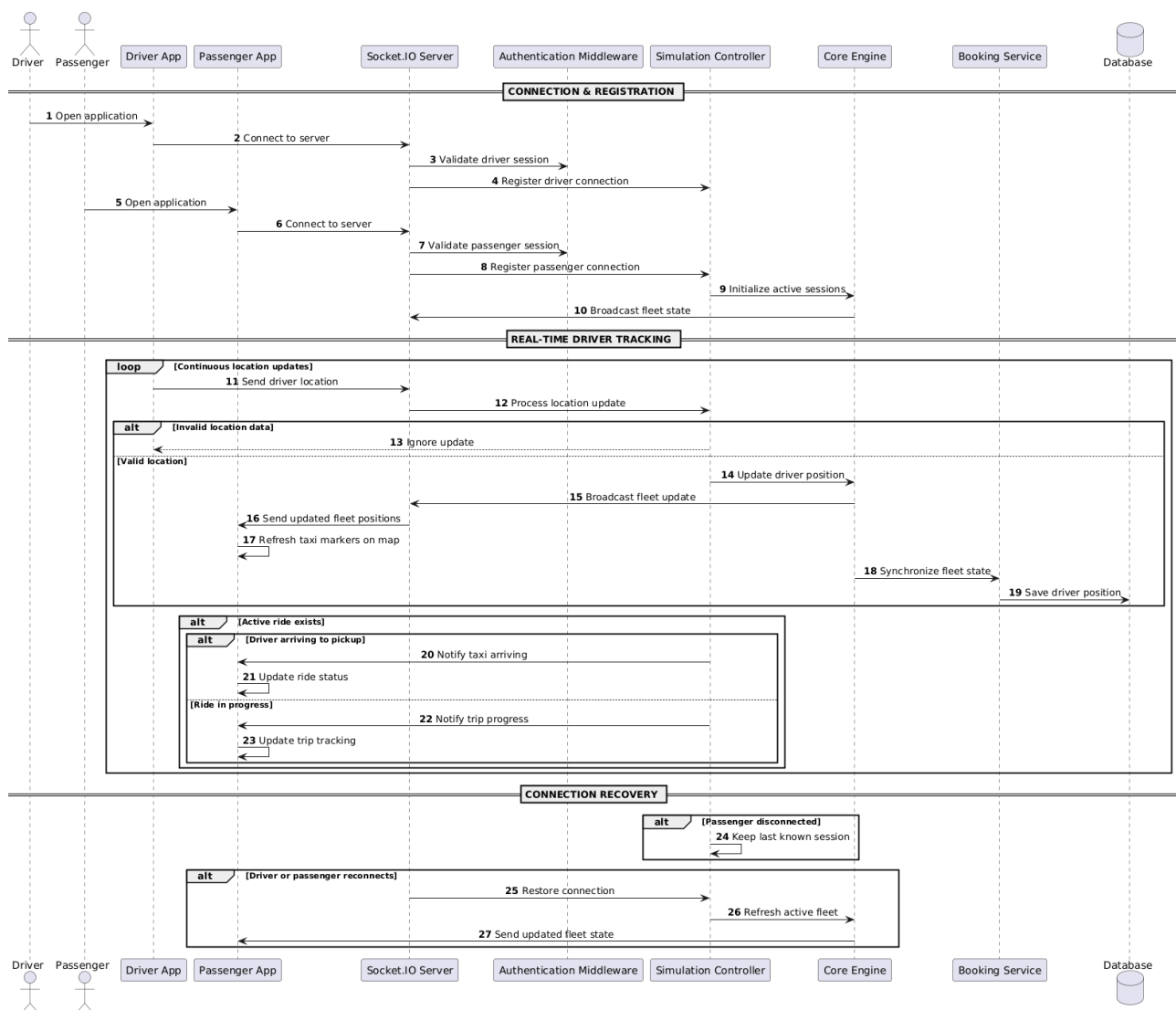


Figure 3.5: Sequence Diagram: Real-Time Driver Tracking

Description of the "Real-Time Driver Tracking" sequence diagram:

The table below describes the real-time tracking process where both driver and passenger are connected to the system through Socket.IO. The system continuously receives driver location updates, processes them through the simulation controller and core engine, and broadcasts updated positions to passengers. It also handles reconnection and session recovery scenarios.

TABLE 3.5 – Description of the Sequence Diagram "Real-Time Driver Tracking"

Title	Real-Time Driver Tracking
Summary	This sequence describes the real-time tracking mechanism of drivers. The system processes continuous GPS updates, synchronizes fleet data, and broadcasts driver positions to passengers using Socket.IO. It also manages session initialization and reconnection scenarios.
Actors	• Driver • Passenger
Preconditions	Both driver and passenger are connected to the system and have valid authenticated sessions.
Main Scenario	1. The driver connects to the system and starts sending location updates. 2. The passenger connects and subscribes to tracking updates. 3. The system validates and processes incoming GPS data. 4. The driver position is updated in the core engine. 5. The system broadcasts updated fleet positions. 6. The passenger receives real-time location updates. 7. The passenger interface refreshes the map markers. 8. The system synchronizes tracking data with the database. 9. If a ride is active, the system sends ride progress updates.
Alternative Scenario	1. Invalid GPS data → update is ignored. 2. Passenger disconnected → last session state is preserved. 3. Driver or passenger reconnects → system restores active tracking state.
Postconditions	Real-time driver positions are continuously synchronized between driver and passenger applications.

3.2.6 Admin/Agent Manage Drivers Sequence Diagram

The figure below presents the "Manage Drivers" sequence diagram:

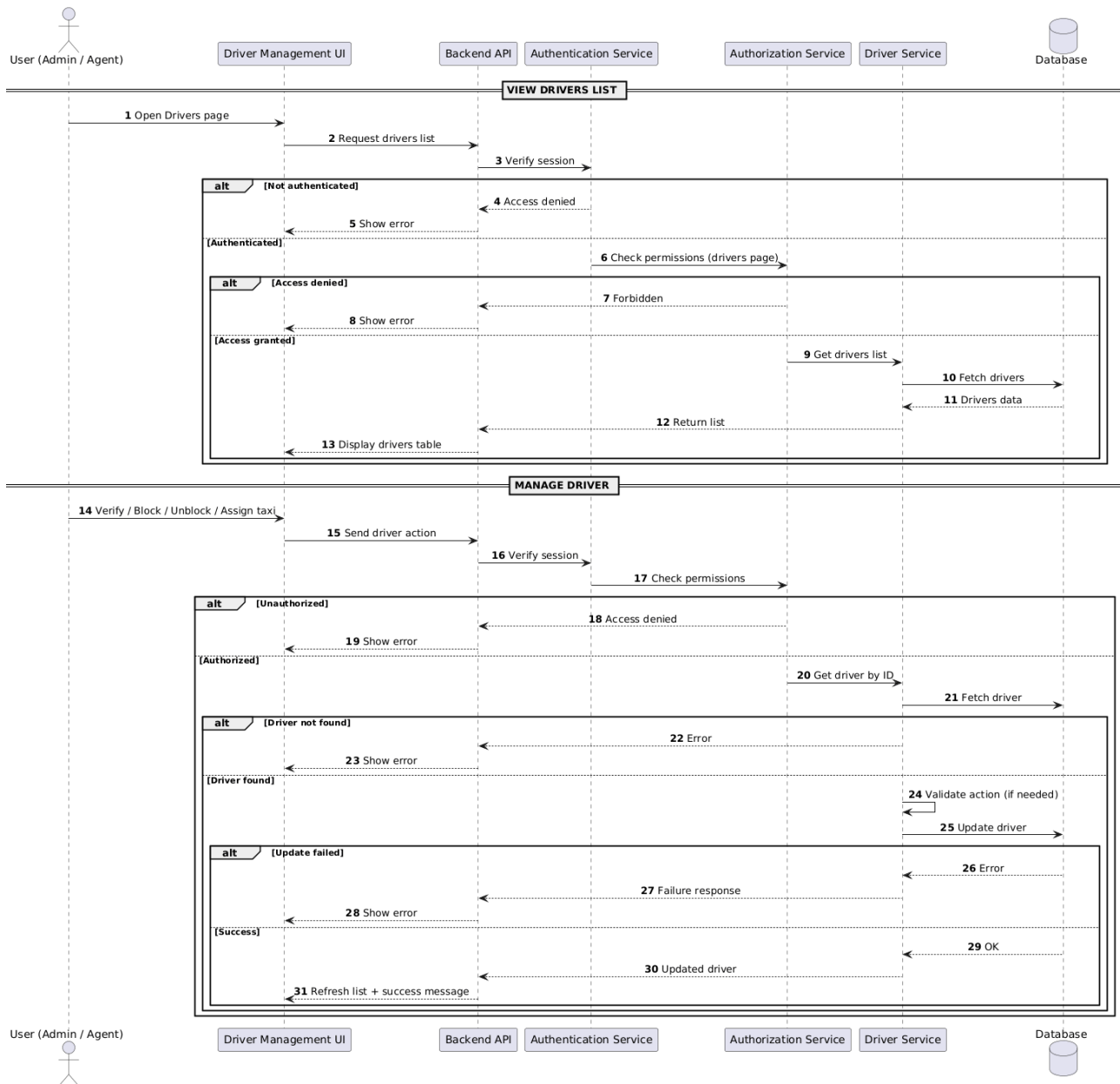


Figure 3.6: Sequence Diagram: Manage Drivers

Description of the "Manage Drivers" sequence diagram:

The table below describes the process followed by an administrator or agent to view and manage drivers in the system. The system verifies authentication, checks permissions, retrieves driver data, and executes management actions such as verification, blocking, or updating driver information.

TABLE 3.6 – Description of the Sequence Diagram "Manage Drivers"

Title	Manage Drivers
Summary	Administrators and agents can view the list of drivers and perform management actions such as verification, blocking, unblocking, or assigning taxis. The system ensures authentication and authorization before executing any operation.

Title	Manage Drivers
Actors	• Admin • Agent
Preconditions	The user must be authenticated and have the required permissions to access the driver management module.
Main Scenario	1. The admin or agent opens the drivers management page. 2. The system verifies the user session. 3. The system checks user permissions. 4. The system retrieves the list of drivers from the database. 5. The system displays the drivers table. 6. The user selects an action (verify, block, unblock, or update driver). 7. The system validates the selected action. 8. The system updates driver information in the database. 9. The system refreshes the driver list with updated data.
Alternative Scenario	1. The user is not authenticated → access is denied. 2. The user does not have sufficient permissions → operation is rejected. 3. Driver ID is not found → error message is displayed. 4. Database update fails → system returns an error and no changes are applied.
Postconditions	• Driver data is updated successfully. • Changes are reflected in real-time in the system. • Access control rules are enforced.

3.3 Conclusion

This chapter presented the design phase of the proposed system through a set of sequence diagrams describing the main interactions between users and system components. These diagrams helped model the dynamic behavior of the application, including authentication, ride management, real-time tracking, and administrative operations. The next chapter will focus on the implementation and realization of the proposed solution.

Table of Contents

4.1	Introduction	46
4.2	Working Environment	46
4.2.1	Hardware Environment	46
4.2.2	Software Environment	46
4.3	System Architecture	50
4.3.1	Physical Architecture	50
4.3.2	Logical Architecture	51
4.4	Test Cases (Experimental Results)	52
4.4.1	Introduction	52
4.4.2	Passenger Interface	52
4.4.3	Driver Interface	53
4.4.4	Ride Request Notification Interface	55
4.4.5	Admin / Agent Dashboard	55
4.4.6	Analytics Dashboard Interface	57
4.4.7	Real-Time Tracking Interface	58
4.5	Conclusion	58

4.1 Introduction

This chapter presents the implementation phase of the proposed system. It describes the working environment, the technologies used, and the tools adopted during the development process.

It also highlights the system architecture and illustrates the main functionalities through screenshots and execution scenarios. The objective is to provide a clear understanding of how the theoretical design was transformed into a functional real-time taxi booking simulation platform.

4.2 Working Environment

This section presents the technical environment used for the development and execution of the application. It includes the software tools, hardware configuration, frameworks, and programming languages adopted in the project.

4.2.1 Hardware Environment

Our project is implemented on the hardware structure described in Table 4.1 :

TABLE 4.1 – Description of the hardware environment

Laptop	MSI Katana 15 B13V
Processor	Intel Core i5-13420H (13th Gen)
RAM	16 Go
Hard Drive	512 Go SSD
Operating System	Windows 11

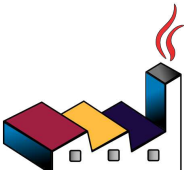
4.2.2 Software Environment

Throughout our project, we use various tools according to the needs of each phase, including design, development, testing, and report writing. In the following sections, we present and describe these tools in detail.

4.2.2.1 Design Tools

The tool used for design purposes is presented in Table 4.2 :

TABLE 4.2 – Design Tools

Tool	Description
	PlantUML is a modeling tool that allows the generation of UML diagrams from simple text-based descriptions. It provides flexibility, fast diagram creation, and easy integration with various development environments. In our project, PlantUML was used to simplify the design and maintenance of UML diagrams without relying on complex graphical tools. [1].

4.2.2.2 Development Tools

Development tools are presented in Table 4.3.

TABLE 4.3 – *Development Tools*

Tool	Description
	Visual Studio Code is a free, cross-platform code editor available on Windows, Linux, and macOS. It supports multiple programming languages and provides a wide range of features useful for developers, such as extensions, debugging tools, and integrated terminal support. It is widely used by developers due to its simplicity, flexibility, and rich ecosystem that covers most development needs. [2].

4.2.2.3 Version Control and Collaboration Tools

The version control, collaboration, and code management tool is presented in Table 4.4.

TABLE 4.4 – *Version Control and Collaboration Tools*

Tool	Description
	GitLab is a web-based source code management and collaboration platform built on Git. It enables code versioning, project management, and teamwork through integrated features such as CI/CD pipelines, issue tracking, and merge requests.[3].

4.2.2.4 Testing Tools

The tools used to perform the necessary tests to ensure the proper functioning of our application are presented in Table 4.5:

TABLE 4.5 – *Testing Tools*

Tool	Description
 POSTMAN	Postman is a powerful tool used for developing, testing, and documenting web services. In our project, it is used to perform functional testing of the backend APIs by validating their correct behavior and verifying the structure of responses before integrating them into the frontend application [4].

4.2.2.5 Report Writing Tools

This report was written using Overleaf, which is described in Table 4.6.

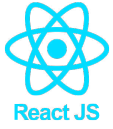
TABLE 4.6 – Report Writing Tools

Tool	Description
	Overleaf is a free collaborative LaTeX editor that provides ready-to-use templates for various needs such as CVs, reports, and more, with easy document sharing. It allows efficient management of document layout and structure without the need for any local installation. [5].

4.2.2.6 Programming Languages and Frameworks

The programming languages and frameworks used throughout the development phase are presented in the following table. 4.7 :

TABLE 4.7 – Programming Languages and Frameworks

Tool	Description
	JavaScript runtime environment used to build the backend server. It allows handling multiple real-time connections efficiently, which is essential for managing ride requests and driver communication. [6].
	Express.js is a lightweight backend framework for Node.js used to build web applications and REST APIs. It simplifies server-side development by handling routing and middleware. In this project, it is used to manage authentication, simulation endpoints, and administrative operations. [7].
	React.js is a JavaScript library used to build interactive user interfaces. It is based on reusable components and uses the Virtual DOM to improve performance. In this project, it is used to develop the web simulation dashboard and admin interface. [8].
	React Native is a framework used to build mobile applications using JavaScript and React. It allows developing for both Android and iOS with a single codebase. In this project, it is used for the passenger and driver mobile applications. [9].
	JavaScript is a programming language mainly used for web development. It allows creating dynamic and interactive applications. In this project, it is used across both frontend and backend to implement system logic and real-time communication. [10].
	TypeScript is an extension of JavaScript that adds static typing. It improves code reliability and helps detect errors early. In this project, it is used in some frontend parts to enhance maintainability. [11].

Tool	Description
	User interfaces are built using React, which relies on HTML and CSS concepts for rendering and styling components. [12].

4.2.2.7 APIs and Libraries Used

In this section, we list all the APIs and libraries used in Table 4.8:

TABLE 4.8 – APIs and Libraries Used

Tool	Description
	Socket.IO is a JavaScript library for web applications that enables real-time bidirectional communication between clients and servers. It uses WebSockets and other transport mechanisms to ensure efficient and continuous data exchange between system components.[13].
	OSRM (Open Source Routing Machine) is an external routing service used to compute real-world distances, estimated travel times, and route geometry between pickup and drop-off locations. It enhances the accuracy of trip simulation by providing realistic road-based navigation data. [14].
	JSON Web Token (JWT) is an open standard used for authentication and secure information exchange between a client and a server. It is compact and self-contained, allowing user data to be transmitted securely without server-side session storage. JWT enables stateless authentication, improving system performance and scalability. [15].
	Leaflet is a JavaScript library used in the web client to display interactive maps and visualize taxi positions and routes in real time. It provides tools for map rendering, marker management, and geographic visualization. [16].
	Mapbox is a mapping platform used in mobile applications to provide advanced map rendering, geolocation services, and route visualization. It enables accurate positioning of users and vehicles, as well as smooth display of routes in real time.[17].
	Bcrypt is a password-hashing library used to securely store user credentials. It applies strong cryptographic hashing with salting to protect passwords and enhance authentication security. [18].

4.2.2.8 Database

The database tool used in our project is presented in Table 4.9.

TABLE 4.9 – Database

Tool	Description
 mongoDB.	MongoDB is used as the main NoSQL database to store application data such as users, bookings, drivers, and ride history. It provides flexibility for handling dynamic data structures and supports scalability for real-time applications. [19]

4.3 System Architecture

4.3.1 Physical Architecture

It is essential in any software design process to select an appropriate architecture in order to ensure good system performance, maintainability, and scalability.

Our application is based on a layered (n-tier) architecture, which clearly separates system responsibilities into independent levels. This separation improves maintainability, extensibility, and overall system understanding.

The system is structured into four main layers:

- **Presentation Layer:** This layer represents the user interface of the system. It includes the passenger and driver mobile applications, as well as the administrative and agent web dashboards. It is responsible for user interaction and sending requests to the backend through REST APIs and real-time communication using Socket.IO.
- **Business Layer:** This layer represents the core functionality of the system. It is implemented using Node.js and Express.js and contains all business logic. It manages essential functionalities such as authentication, ride booking management, driver dispatching, trip simulation, and real-time tracking. It also includes internal components such as the Simulation Controller, Core Engine, and the intelligent dispatching module.
- **Data Access Layer:** This layer is responsible for communication with the MongoDB database. It handles data retrieval, insertion, update, and deletion operations for users, drivers, bookings, and ride history.
- **Domain Layer (Entity Layer):** This layer defines the core business entities of the system such as User, Driver, Booking, and Ride. It provides a structured representation of the data used across the application.

This architecture improves system organization by separating responsibilities, which simplifies development, testing, and maintenance.

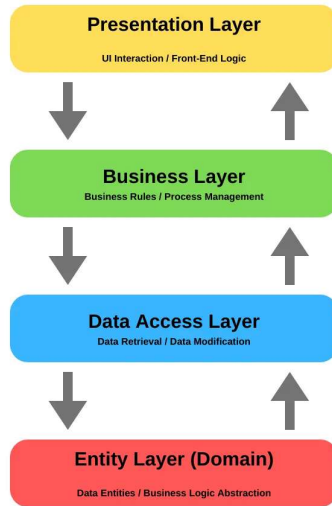


Figure 4.1: Layered (n-tier) architecture of the system

4.3.2 Logical Architecture

The logical architecture describes the organization of software components and their interactions within the system.

The system is mainly composed of two parts:

- **Front-End:** Developed using React and React Native, it provides interactive user interfaces for passengers, drivers, and administrators.
- **Back-End:** Developed using Node.js and Express.js, it handles business logic, REST APIs, and real-time communication through Socket.IO.

The figure below illustrates a simplified view of the system architecture, showing the interaction between the frontend, backend, and database layers.

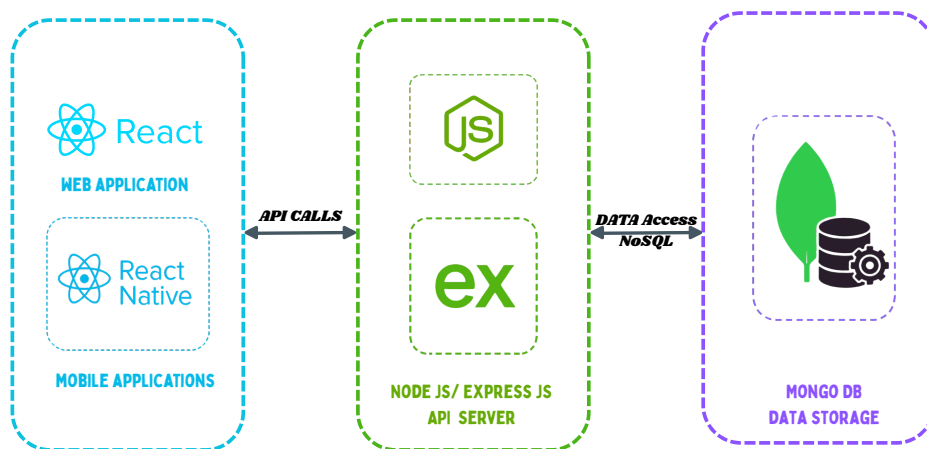


Figure 4.2: Logical architecture of the system

4.4 Test Cases (Experimental Results)

4.4.1 Introduction

This section presents the test cases and execution results of the developed application. It aims to validate the correct functioning of the system by illustrating the main user interfaces and key scenarios of the platform, including passenger, driver, and administrative operations.

4.4.2 Passenger Interface

The passenger application allows users to request rides, track drivers in real time, and manage their trips.

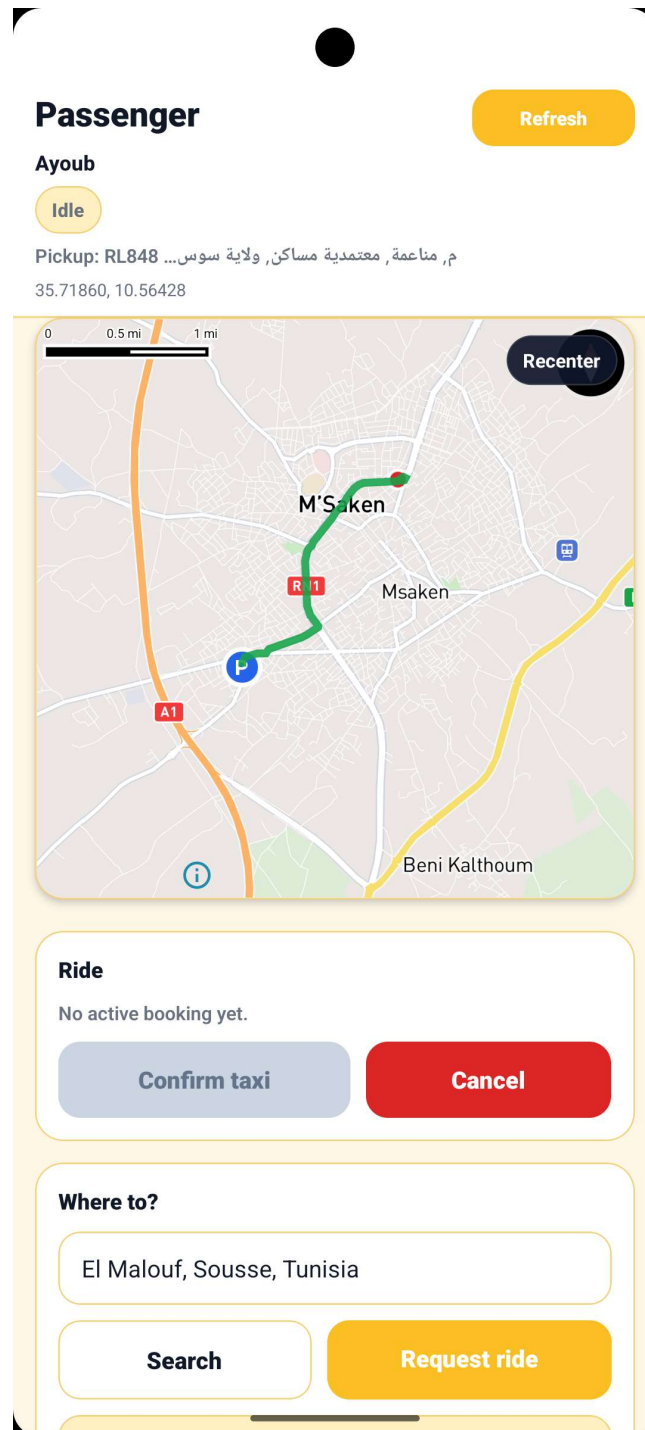


Figure 4.3: Passenger Home Interface

Description: This interface allows the passenger to enter pickup and destination locations and request a ride. The system then processes the request and assigns an available driver.

4.4.3 Driver Interface

The driver interface enables drivers to receive ride requests and manage trip execution.

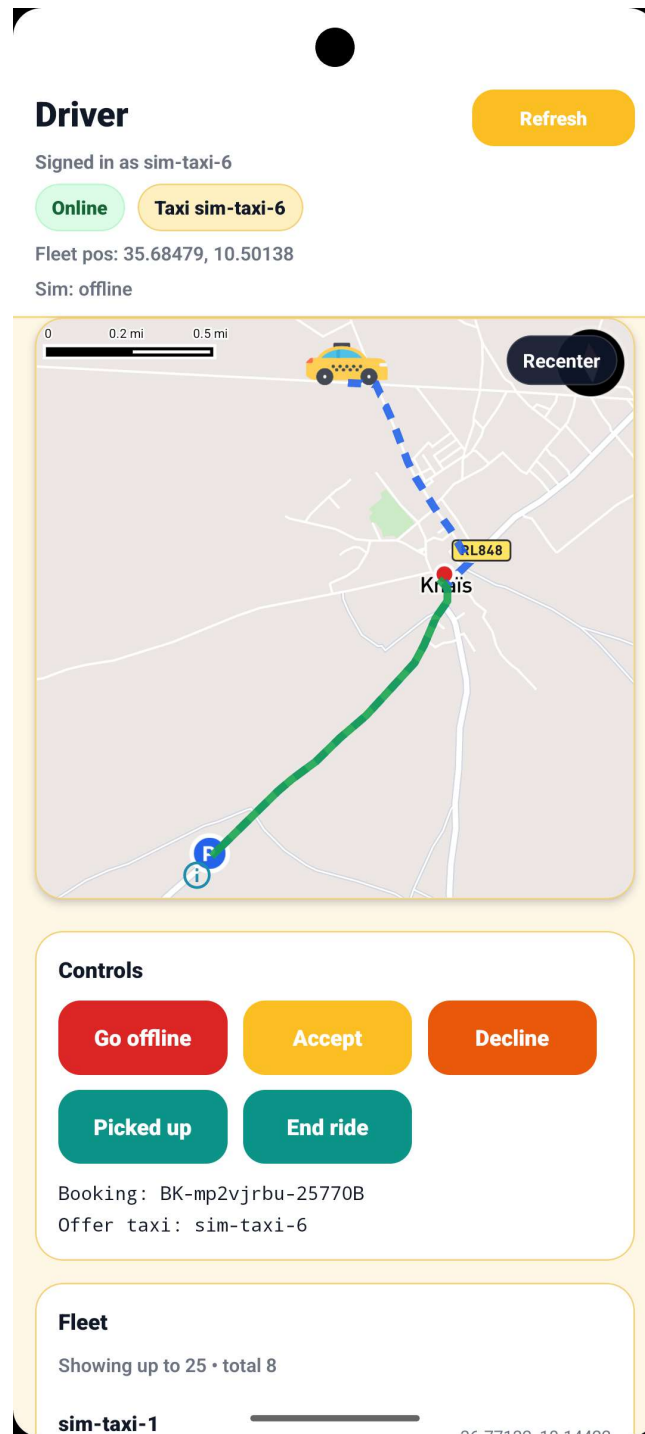


Figure 4.4: Driver Dashboard Interface

Description: This screen shows incoming ride requests. The driver can accept or reject a request, and once accepted, the trip status is updated in real time.

4.4.4 Ride Request Notification Interface

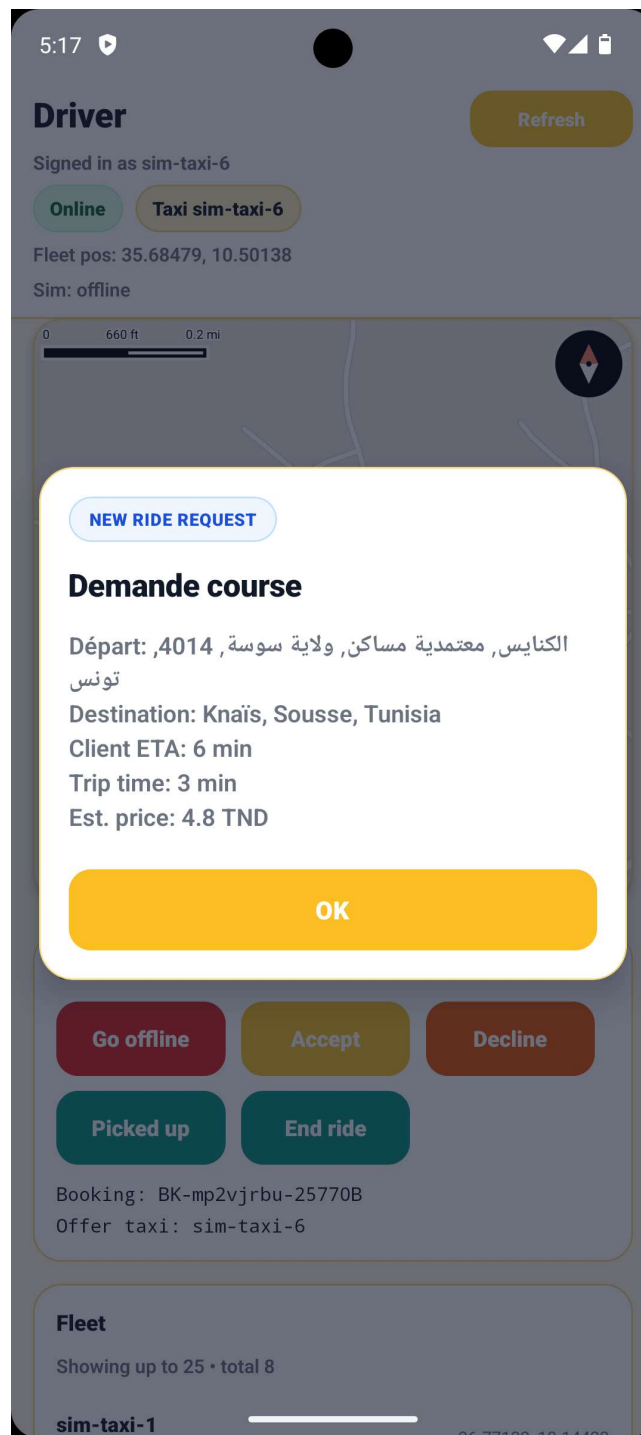


Figure 4.5: Driver Ride Request Notification

Description: This interface shows the real-time ride request notification received by the driver after a passenger submits a booking request. The driver can accept or reject the ride directly from the mobile application. The request includes trip information such as pickup location, destination, and estimated distance.

4.4.5 Admin / Agent Dashboard

Dashboard interface is used to manage system users, especially drivers.

Drivers

4 total

Manage drivers and fleet.

Refresh

All 4

Verified 4

Unverified 0

Blocked 1

Active 3

NAME	EMAIL ↑↓	TAXI CODE ↑↓	VERIFIED ↑↓	ACCOUNT ↑↓	CREATED	
Ayoub Ben Salah	sande@taxi.tn	sim-taxi-8	Yes	Active	5/1/2026, 10:44:04 AM	...
Firas Moumen	test@taxi.tn	sim-taxi-6	Yes	Active	4/22/2026, 5:21:22 PM	...
Sami Bouazizi	test2@taxi.tn	sim-taxi-2	Yes	Blocked	4/22/2026, 5:21:33 PM	...
Karim Haddad	test1@taxi.tn	sim-taxi-4	Yes	Active	4/22/2026, 5:21:29 PM	...

Figure 4.6: Driver Management Interface

Description: This interface allows administrators and agents to view driver information, verify accounts, block or unblock drivers, and manage driver-related operations.

4.4.6 Analytics Dashboard Interface

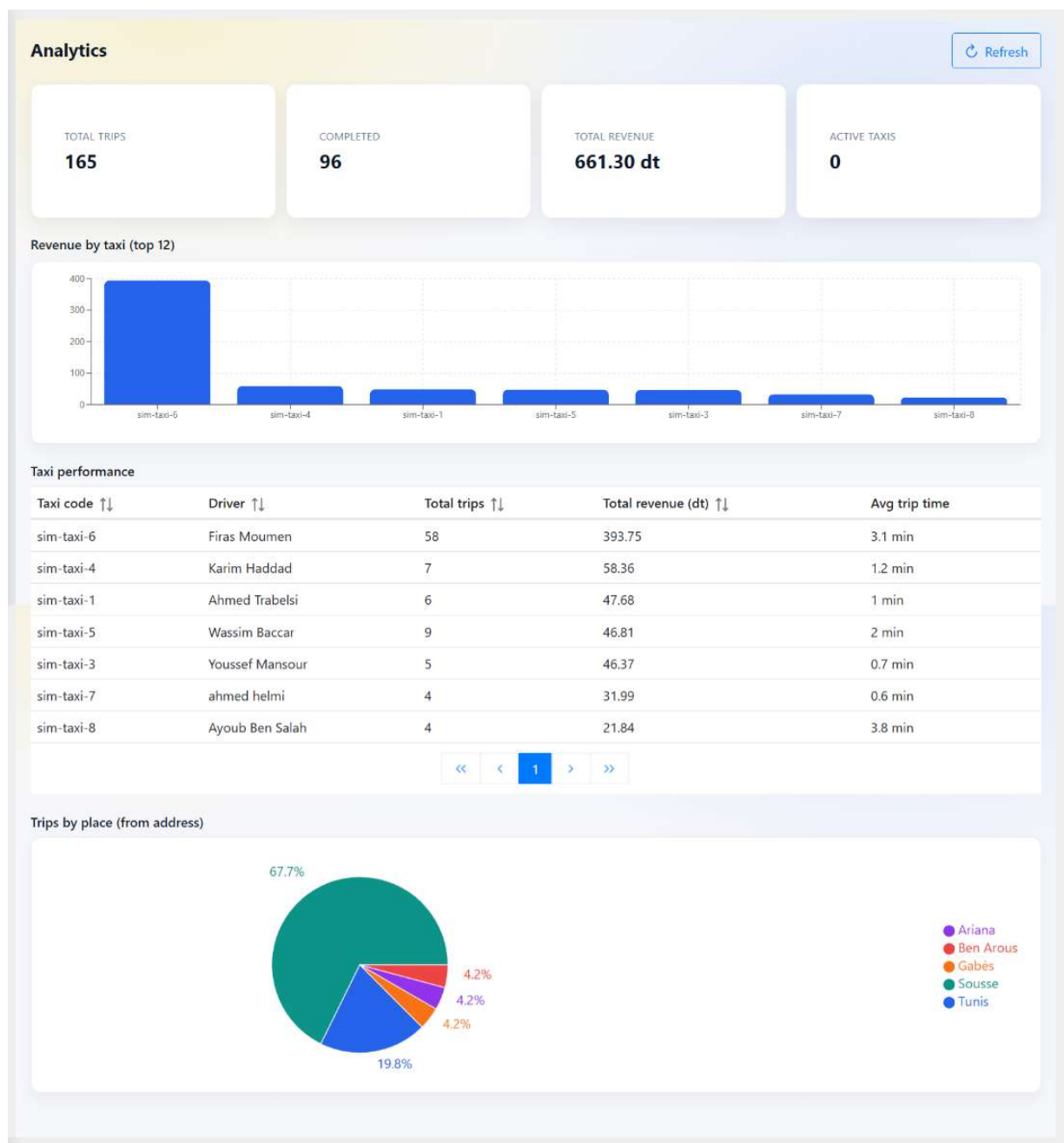


Figure 4.7: Analytics Dashboard Interface

Description: This dashboard provides analytical and statistical information about the taxi simulation platform, including key performance indicators such as the total number of trips, completed rides, generated revenue, active taxis, driver performance statistics, revenue distribution by taxi, average trip duration, and trip distribution by geographic area, allowing administrators to monitor system activity and evaluate operational performance in real time.

4.4.7 Real-Time Tracking Interface

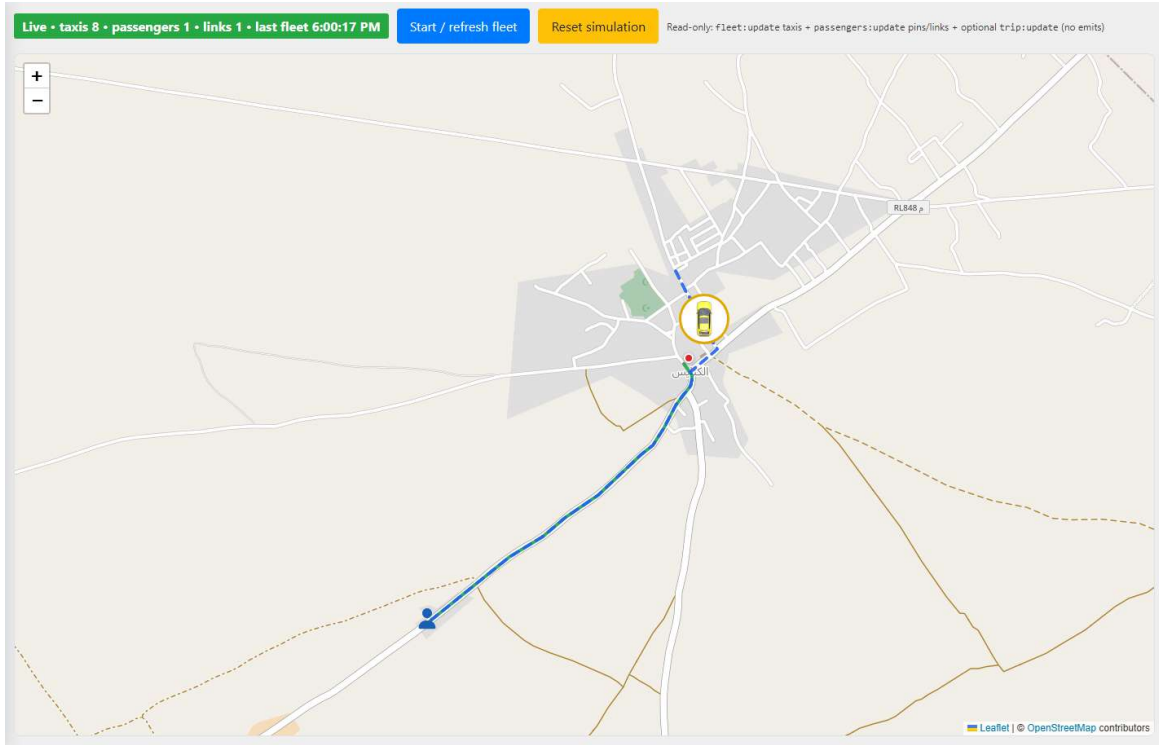


Figure 4.8: Real-Time Driver Tracking Interface

Description: This interface shows live tracking of drivers on the map using Socket.IO and real-time GPS updates.

4.5 Conclusion

In this chapter, we presented the implementation phase of the proposed taxi booking simulation platform. We first introduced the working environment, including the hardware and software tools used throughout the development process. Then, we described the adopted system architecture and the interaction between the different application components.

Finally, we illustrated the main functionalities of the platform through several interfaces and execution scenarios, including authentication, ride management, real-time tracking, driver management, and analytics monitoring. These implementation results demonstrate the effectiveness of the proposed solution and validate the system requirements defined during the analysis and design phases.

General Conclusion

THIS REPORT WAS CARRIED OUT AS PART OF OUR END-OF-STUDIES PROJECT FOR THE APPLIED LICENSE IN COMPUTER ENGINEERING AT THE HIGHER INSTITUTE OF APPLIED SCIENCES AND TECHNOLOGY OF SOUSSE (ISSAT SOUSSE). IT WAS DEVELOPED DURING A PROFESSIONAL INTERNSHIP AT THE COMPANY "SMARTEO", WHERE WE HAD THE OPPORTUNITY TO WORK ON THE REALIZATION OF THIS PROJECT.

The objective of this project was the design and implementation of a real-time taxi booking simulation platform capable of managing passenger requests, driver interactions, ride dispatching, and live trip tracking through modern web and mobile technologies.

Throughout this work, we analyzed the functional and technical requirements of the system and designed a layered architecture adapted to real-time communication and scalable application development. The project was developed progressively through several phases, including requirement analysis, system design, implementation, and testing.

The developed platform includes several important functionalities such as secure authentication using JWT, passenger and driver mobile applications, real-time communication using Socket.IO, ride request management, intelligent driver dispatching, live driver tracking, administrative management interfaces, and analytics dashboards for operational monitoring.

This project allowed us to strengthen our technical knowledge and improve our practical skills in full-stack development, API integration, real-time systems, software architecture, and database management. In addition, it helped us develop organizational and communication skills through project planning, problem solving, and collaborative work.

The technologies used during this project include React Native, React.js, Node.js, Express.js, MongoDB, Socket.IO, and OSRM, which provided an efficient environment for building a dynamic and scalable real-time application.

Although the current version of the platform satisfies the main objectives of the project, several future improvements can still be considered in order to enhance the system further, such as:

- Integration of an online payment system.
- Implementation of AI-based ride prediction and intelligent dispatch optimization.
- Addition of push notification services for ride updates and alerts.
- Integration of a driver and passenger rating system.
- Deployment of the platform using cloud infrastructure for scalability and high availability.

In conclusion, this project represents a valuable professional and academic experience that allowed us to apply theoretical knowledge in a real-world context while developing a complete real-time transportation simulation platform.

Bibliographie

- [1] *PlantUML Official Website*. URL: <https://plantuml.com/> (visited on 03/08/2026).
- [2] *Visual Studio Code*. URL: <https://code.visualstudio.com/> (visited on 04/14/2026).
- [3] *GitLab Official Website*. URL: <https://gitlab.com/> (visited on 03/21/2026).
- [4] *Postman*. URL: <https://www.postman.com/> (visited on 04/02/2026).
- [5] *Overleaf*. URL: <https://fr.overleaf.com/about> (visited on 03/29/2026).
- [6] *Node.js Official Website*. URL: <https://nodejs.org/> (visited on 04/18/2026).
- [7] *Express.js Official Website*. URL: <https://expressjs.com/> (visited on 03/12/2026).
- [8] *React Official Website*. URL: <https://react.dev/> (visited on 04/25/2026).
- [9] *React Native Official Website*. URL: <https://reactnative.dev/> (visited on 03/05/2026).
- [10] *JavaScript Documentation (MDN)*. URL: <https://developer.mozilla.org/en-US/docs/Web/JavaScript> (visited on 04/09/2026).
- [11] *TypeScript Official Website*. URL: <https://www.typescriptlang.org/> (visited on 03/17/2026).
- [12] *HTML & CSS Documentation (MDN)*. URL: <https://developer.mozilla.org/> (visited on 04/11/2026).
- [13] *Socket.IO Official Website*. URL: <https://socket.io/> (visited on 03/26/2026).
- [14] *Open Source Routing Machine (OSRM)*. URL: <http://project-osrm.org/> (visited on 04/06/2026).
- [15] *JSON Web Tokens (JWT)*. URL: <https://jwt.io/> (visited on 03/14/2026).
- [16] *Leaflet.js Official Website*. URL: <https://leafletjs.com/> (visited on 04/20/2026).
- [17] *Mapbox Official Website*. URL: <https://www.mapbox.com/> (visited on 03/23/2026).
- [18] *Bcrypt Library Documentation*. URL: <https://www.npmjs.com/package/bcrypt> (visited on 04/01/2026).
- [19] *MongoDB Documentation*. URL: <https://www.mongodb.com/docs/> (visited on 03/30/2026).